

Databricks Modern Data Platform

Databricks is a unified data management and analytics platform that combines the best features of data warehouses with the flexibility of data lakes, creating a lakehouse architecture. It enables efficient large-scale data processing, analytics, and artificial intelligence development.



Krzysztof Burejza

Azure Data Engineer

Data engineer with 10 years of experience working with data, from SQL Server to cloud platforms like Azure, Databricks, and Microsoft Fabric. Focused on transforming raw data into valuable solutions, building pipelines, data warehouses, and lakehouses. An enthusiast of new technologies at the intersection of data and AI, which make data engineering even more fascinating.



Agenda

Introduction

- Databricks Modern Data Platform
- Krzysztof Burejza (Azure Data Engineer)
- Databricks: Data + AI Platform overview

Data Architecture

- From Tools to Architecture
- End-to-End Thinking About the Data Pipeline
- Logical Layers of a Data Platform
- Separation of Concerns
- Pipeline Structure Pattern: Load → Transform → Serve

Medallion Architecture

- Bronze Layer – Raw Data Preservation
- Silver Layer – Validated & Enriched
- Slowly Changing Dimensions (SCD Type 1, 2, 3, 4)
- Gold Layer – Business-Ready Data
- Managing Dependencies Between Layers

Data Ingestion

- Batch vs Streaming
- COPY INTO / CTAS
- Auto Loader
- Trigger Modes
- Rescued Data Column
- Lakeflow Connect (SaaS Integration)

Lakeflow Pipelines

- Declarative Approach
- Streaming Tables vs Materialized Views
- Data Flows & CDC APPLY
- Data Quality Expectations
- Schema Evolution
- Compute: Serverless vs Classic

Orchestration

- Lakeflow Jobs (Databricks Workflows)
- Job Structure, Task Types & Dependencies
- Triggers and Parameterization
- Observability, Monitoring & Alerting
- Orchestration Patterns & Anti-Patterns

Key Takeaways

Databricks: Data + AI Platform

What is Databricks?

Databricks is a unified analytics platform built on Apache Spark, created by the founders of this project from UC Berkeley. It provides a comprehensive lakehouse solution, combining:

- Data Engineering – data processing and preparation
- Data Science – advanced analytics and machine learning
- Business Analytics – visualizations and business reports

The platform is available on all major public clouds (AWS, Azure, GCP), allowing for flexible deployments consistent with existing infrastructure.





Founding of Databricks

2013: Databricks is formed

The founders – creators of Apache Spark from UC Berkeley, including Matei Zaharia, Ali Ghodsi, Ion Stoica – decide to commercialize the technology.

Company Mission

To simplify the use of Apache Spark in production and in the cloud. To make advanced analytics accessible to a wider range of organizations.

Databricks was founded by the team that not only created Spark, but continues to actively develop both the engine and the ecosystem around it.

Founding Team



Ali Ghodsi
Co-founder and Chief Executive Officer



Ion Stoica
Co-founder and Executive Chairman



Matei Zaharia
Co-founder and CTO



Patrick Wendell
Co-founder and VP of Engineering



Reynold Xin
Co-founder and Chief Architect



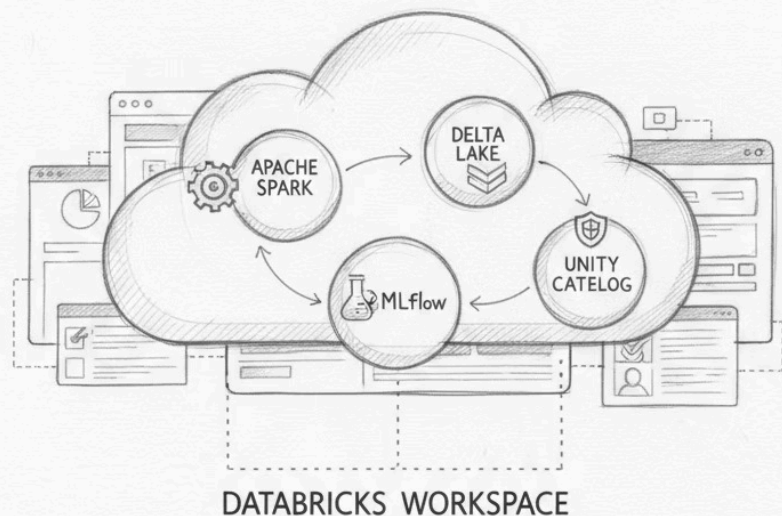
Andy Konwinski
Co-founder and VP of Product Management



Arsalan Tavakoli-Shiraji
Co-founder and SVP of Field Engineering

What is Databricks?

Databricks is a cloud Data + AI platform built on proven open-source technologies:



Apache Spark

Distributed in-memory data processing, the standard big data analytical engine

Delta Lake

Open table format with ACID transactions and time travel, the foundation of Lakehouse

MLflow

Open source platform for managing the ML model lifecycle, from experimentation to deployment

Unity Catalog

Central metadata catalog with advanced governance, lineage, and access control

Available in major public clouds, including Azure Databricks – natively integrated with the Microsoft Azure ecosystem.

Databricks Control vs Data Plane Architecture

Control Plane

Managed by Databricks:

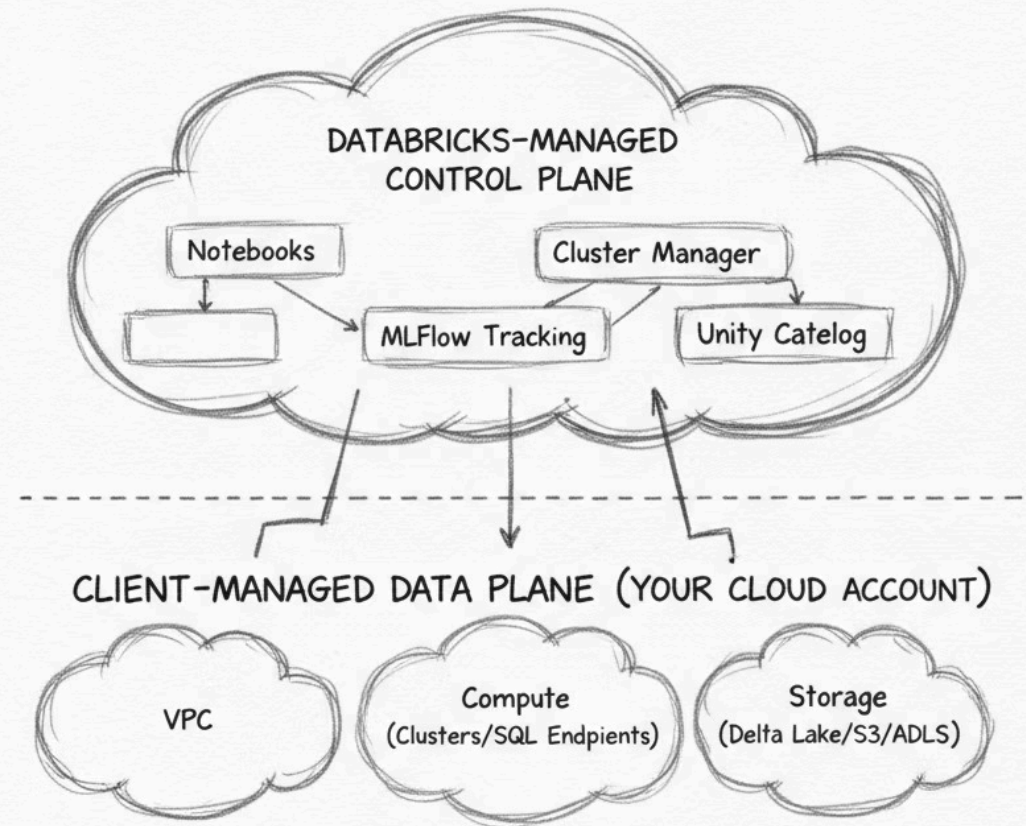
- Web Application – user interface
- Cluster Manager – cluster management
- Jobs Scheduler – job scheduling
- Notebook Server – development environment
- Security & Governance – security
- Metastore – metadata

Data Plane

Customer Environment:

- Compute Clusters (Apache Spark)
- Customer Storage (S3/ADLS/GCS)
- Network (VPC/VNet)
- Security Groups & Policies

Communication between layers occurs via secure API calls, ensuring customer data isolation.



Databricks Cloud Integration

Storage

Native integration with Azure Data Lake Storage, AWS S3, Google Cloud Storage. Databricks reads and writes directly to cloud storage without intermediaries.

Security & Identity

Azure Active Directory, AWS IAM, Google Cloud IAM for authentication. SCIM provisioning and SSO for enterprise. Encryption at rest and in transit.

BI Integrations

Connectors for Power BI, Tableau, Looker, Qlik. JDBC/ODBC drivers for any SQL tools. Partner Connect for quick setup.

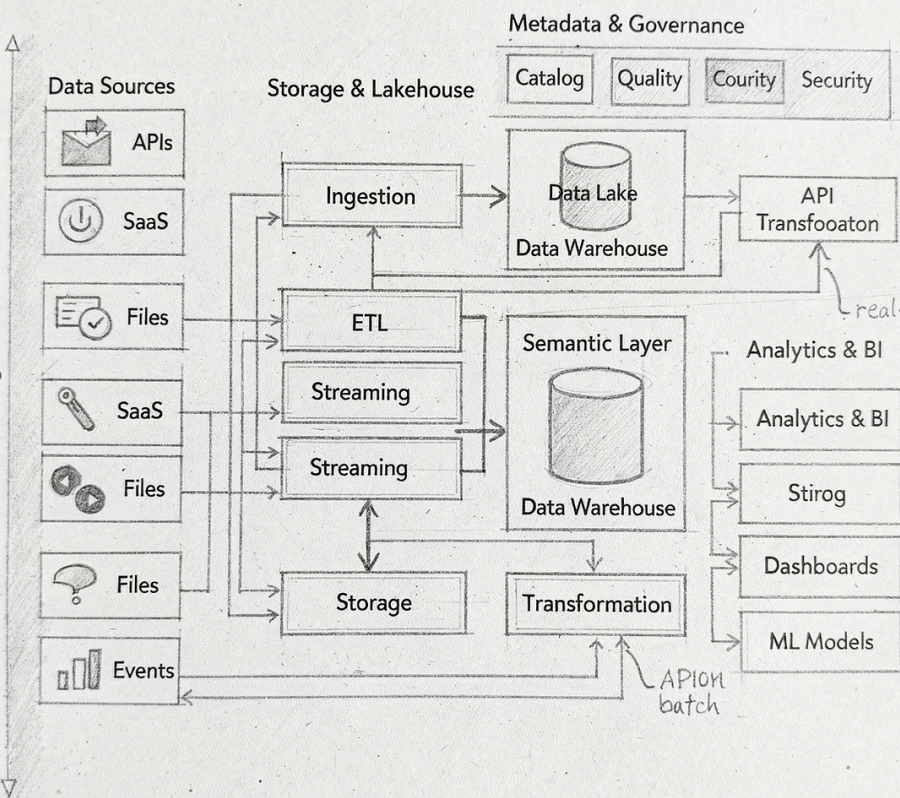
ETL/ELT Integrations

Collaboration with Fivetran, dbt, Azure Data Factory, Informatica, Talend. Auto Loader for incremental file ingestion from cloud storage.



Medallion Architecture

Layered architecture for data



From Tools to Architecture

Auto Loader, Delta Live Tables, and Workflows are powerful components — but by themselves they do not create architecture. Architecture is the way you connect them: what boundaries you define, how you shape data flow, and how you manage dependencies between layers.

Architecture defines the maintainability and scalability of a system. Without it, tools become a source of chaos rather than an advantage.

Why is this important?

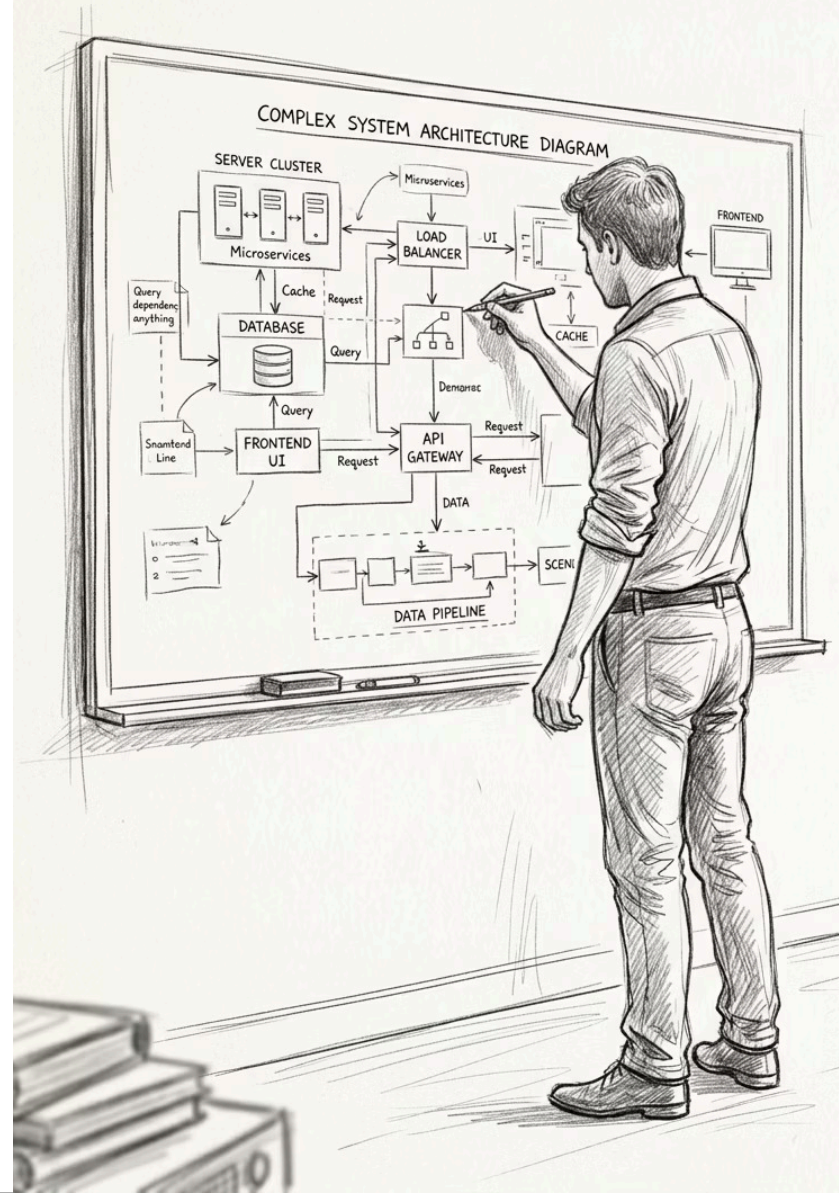
Knowing the tools is only the starting point. The real challenge in data engineering is putting them together into a coherent, scalable system — one that can survive changing requirements, growing volume, and team turnover.

Tooling problem

Auto Loader, DLT, Workflows — we understand each of them separately. But how do we assemble them into a production system?

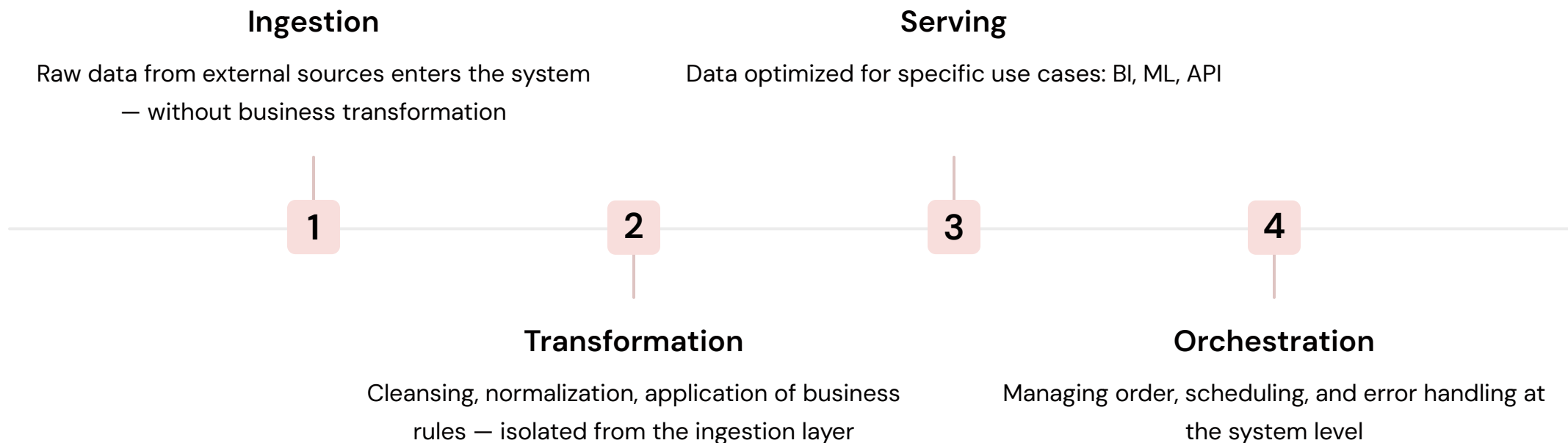
Architectural problem

Production systems require structure, consistency, and clear boundaries of responsibility. Knowing the functions alone is not enough — design matters.



End-to-End Thinking About the Data Pipeline

A data pipeline is not a single task — it is a system of connected stages through which data flows from source to consumer. Each stage has a clearly defined responsibility and delivers data to the next stage in a predictable, repeatable way.



Logical Layers of a Data Platform

A well-designed data platform consists of five clearly separated layers. Each has a different purpose, different requirements, and a different owner. Mixing responsibilities between layers is the most common source of problems in production systems.



Ingestion Layer

Receives data from external sources. Read and write only — zero business logic.



Transformation Layer

Implements business rules, data cleansing, and normalization.



Serving Layer

Optimizes data for specific consumers: reports, ML models, API endpoints.



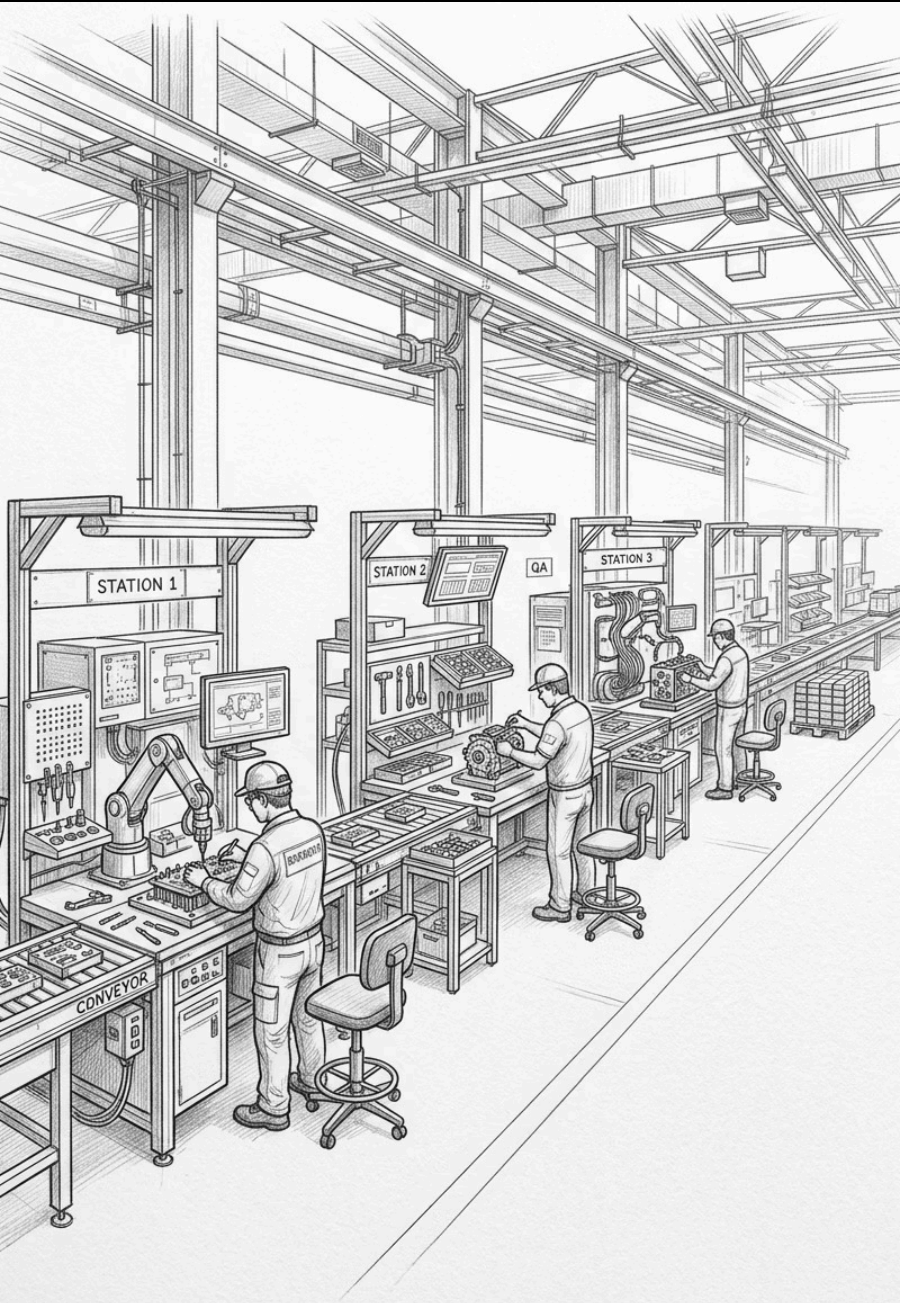
Orchestration Layer

Manages scheduling, ordering, and error handling for the entire system.



Governance Layer

Access control, data lineage tracking, catalog, and quality policies.



Separation of Concerns — Separation of Responsibilities

This is one of the most important principles in system design. In the context of data pipelines, it means: each layer does one thing and does it well. Violating this principle leads to code that is difficult to test, debug, and evolve.

Ingestion without business logic

The Bronze layer stores data exactly as it came from the source. No transformations, no business decisions — only storage and availability guarantees.

Transformation without orchestration dependencies

Transformation logic should be portable — able to run regardless of whether it is triggered by Workflow, DLT, or manually from a notebook.

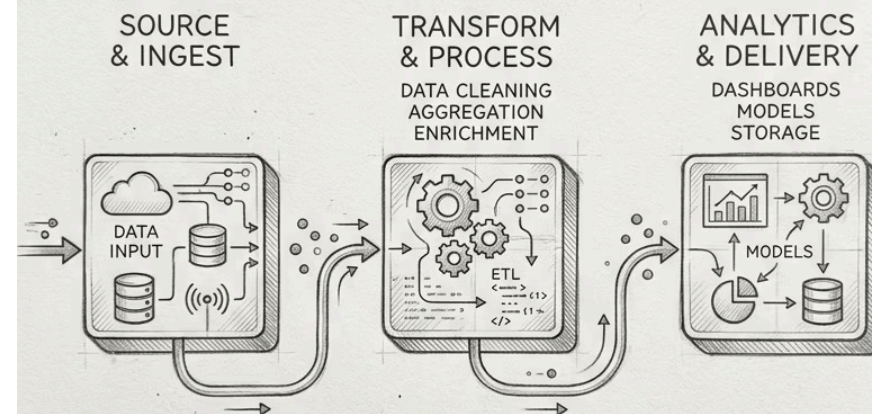
Serving optimized for the consumer

The Gold layer is not a copy of Silver. It is a product designed for a specific use case — with the right partitioning, indexes, and format.

Pipeline Structure Pattern: Load → Transform → Serve

The basic pattern of a production data pipeline consists of three independent steps. Each step is implemented separately, has its own test scope, and can be deployed independently of the others.

Independence between steps is a key property — it enables reuse of transformation logic across different pipelines, simplifies debugging (we know exactly which step failed), and allows each layer to scale independently.



Medallion Architecture

Medallion Architecture is a layered approach to organizing data in a data lake, ensuring structured and gradual data cleansing and transformation. It is crucial for maintaining data quality, governance, and accessibility in complex analytical environments.

Bronze

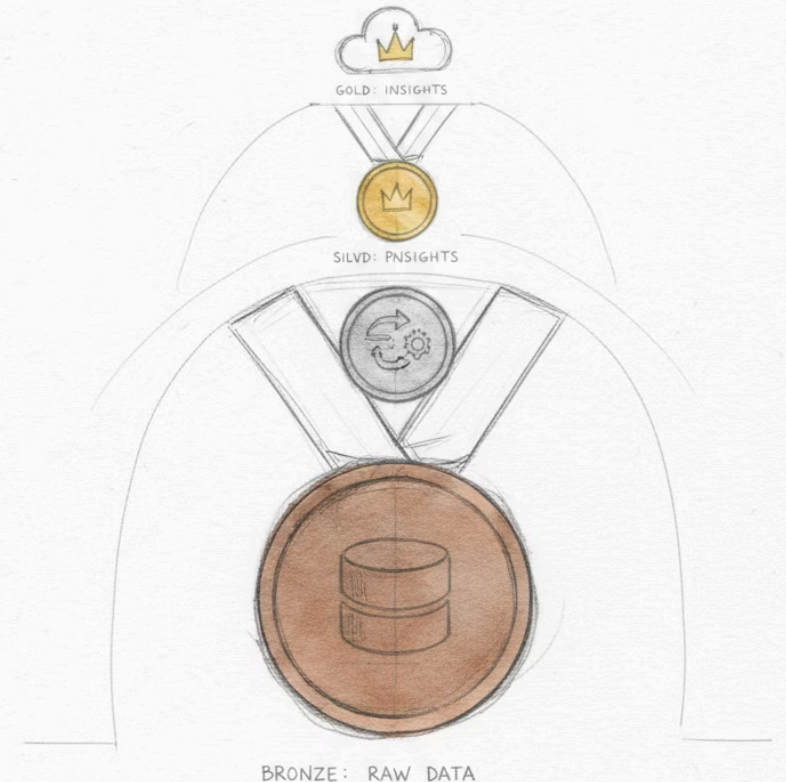
Raw data layer, unchanged from the source.

Silver

Cleaned and validated data layer.

Gold

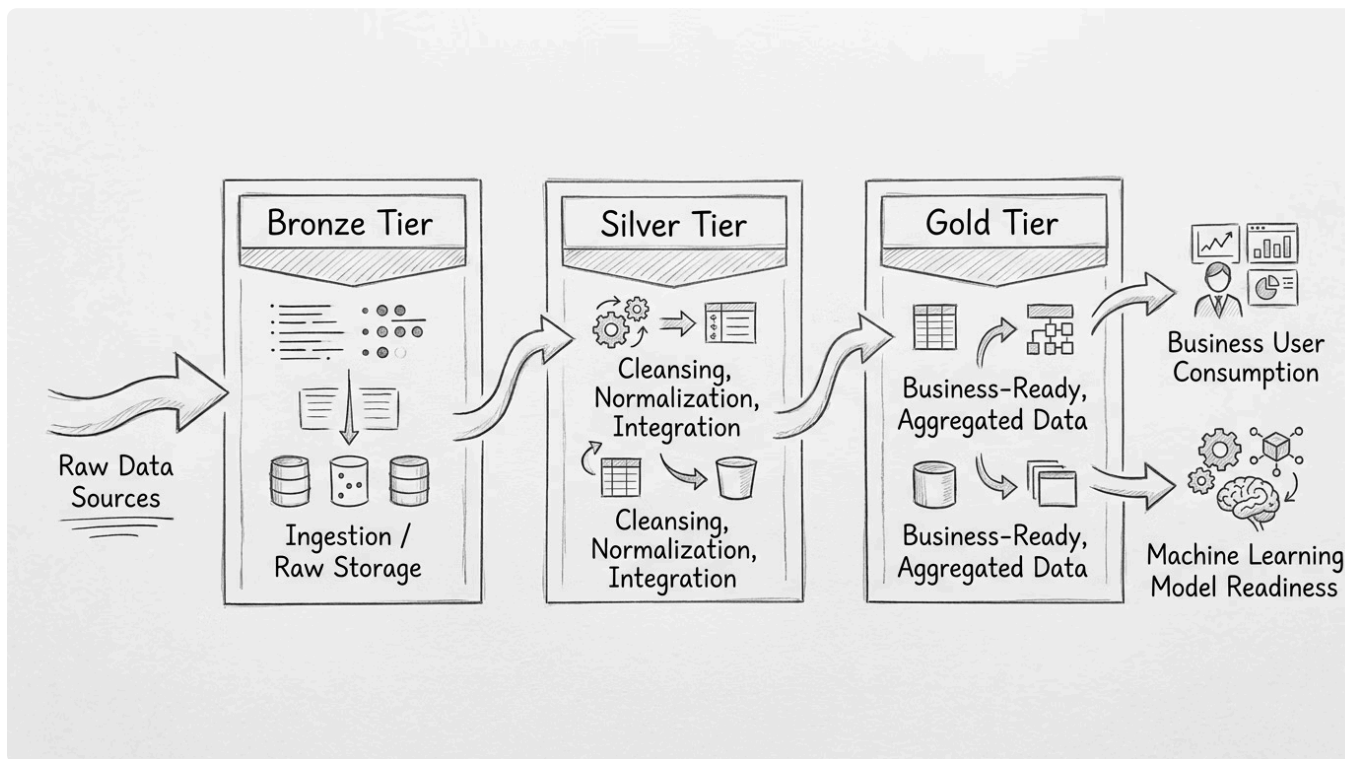
Business-ready data layer for analysis. Data is aggregated and in an easily consumable format.

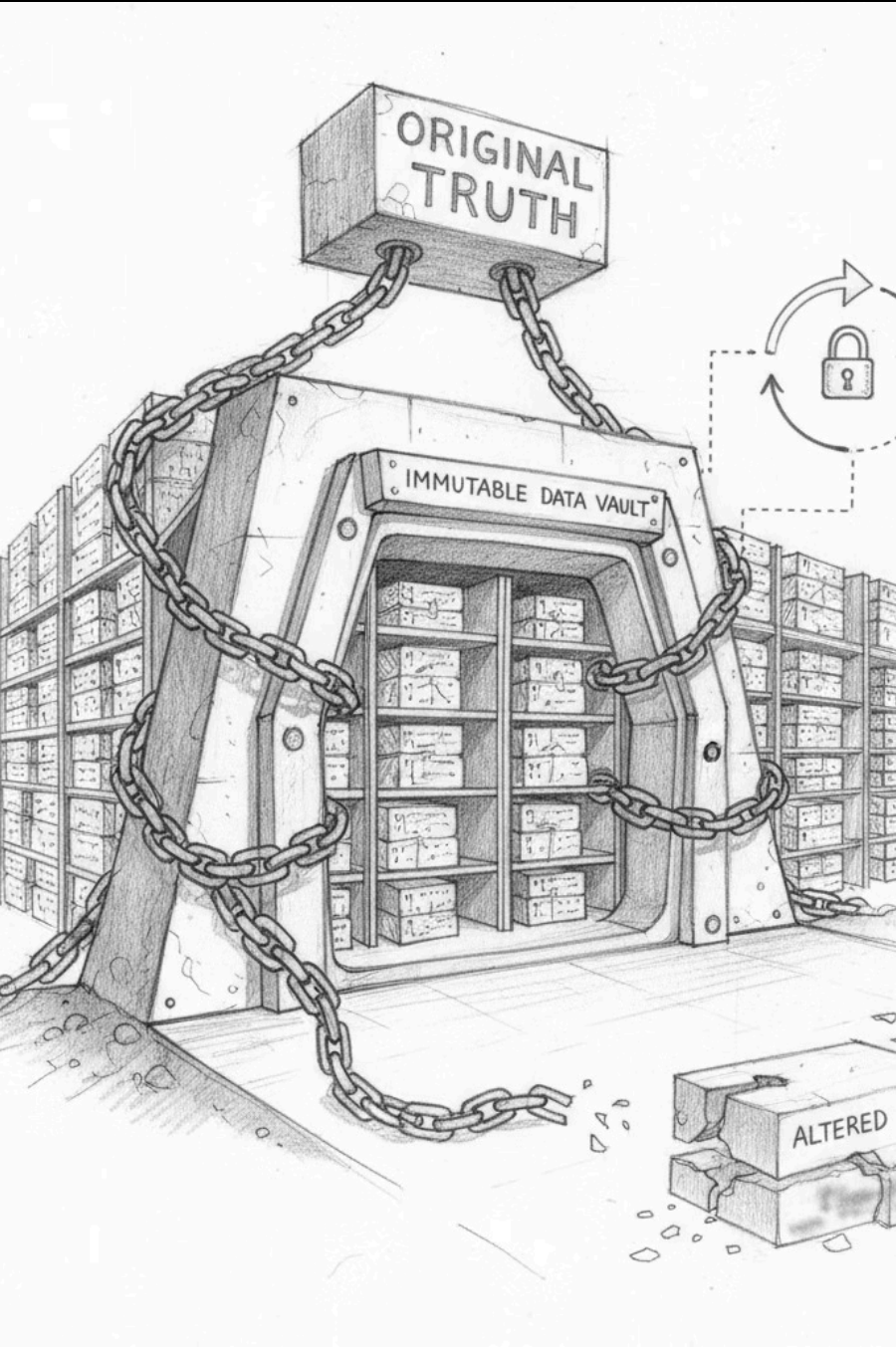


Medallion Architecture

Medallion Architecture is a standard pattern for organizing data in a Lakehouse, which gradually cleanses and enriches raw data into a form ready for consumption by business users and ML models.

Each layer has clearly defined responsibilities and data quality SLAs. Layer separation enables problem isolation, incremental processing, and optimization of storage/compute costs at each stage.





Bronze Layer – Raw Data Preservation

The Bronze layer is an immutable archive of raw data from source systems. Philosophy: **keep everything**, do not transform, add metadata. Bronze is the foundation for auditing, disaster recovery, and re-processing.

Key practices

- Append-only pattern (never DELETE/UPDATE)
- Preservation of original types and formats
- Addition of technical columns (load_timestamp, source_file)
- Utilization of *rescueddata* for bad records
- Retention policy ≥ 90 days (compliance)

```
ALTER TABLE bronze_events
SET TBLPROPERTIES (
  delta.logRetentionDuration = '180 days',
  delta.deletedFileRetentionDuration = '90 days'
);
```



Metadata enrichment

Each record in Bronze should contain:

- `_load_timestamp` – load time
- `_source_file` – source file path
- `_source_system` – system identifier
- `_ingestion_id` – UUID batch/stream job

Bronze Is Not Just Storage — It's a Contract

The Bronze layer serves as a **data contract** between source systems and the rest of the platform. Its main responsibility is to guarantee that source data is available in its unchanged form — now and in the future.

- **Availability guarantee:** data is preserved regardless of source changes
- **Replay and backfill support:** the ability to reprocess historical data without source access
- **Audit trail:** full change history with metadata (ingestion timestamp, source, schema version)
- **Isolation from failures:** source issues do not block processing of already ingested data

Key principle

Never modify data in the Bronze layer. Every change should be a new version or a new record — not an update to an existing one.

Schema Evolution

Bronze should handle source schema changes without failing — use `mergeSchema` or schema evolution mode in Auto Loader.

Silver Layer – Validated & Enriched

Cleansing

Removal of null values, format normalization, trim whitespace, encoding standardization

Deduplication

Removal of duplicates via MERGE INTO or window functions (ROW_NUMBER)

Validation

Business rules, referential integrity checks, range validation, quality flags

Enrichment

Join with dimension tables, calculated fields, currency conversion, geocoding

Silver is the **single source of truth** for downstream consumers. Data in Silver is guaranteed to be high-quality and consistent – if a record exists in Silver, it is business-correct.

Designing Silver as a Business Rules System

Silver is the central point for applying business logic across the entire platform. If a business rule exists in multiple places, that is a sign of a design flaw — not a feature.

Single source of rules

Each transformation rule is defined once and used everywhere. A rule change propagates automatically to all Silver consumers.

Consistent transformations

Normalization, deduplication, and data type harmonization — applied consistently, not ad hoc in every notebook separately.

Enforcing data quality

Schema validation, DLT constraints (EXPECT), and rejecting or quarantining records that do not meet requirements.

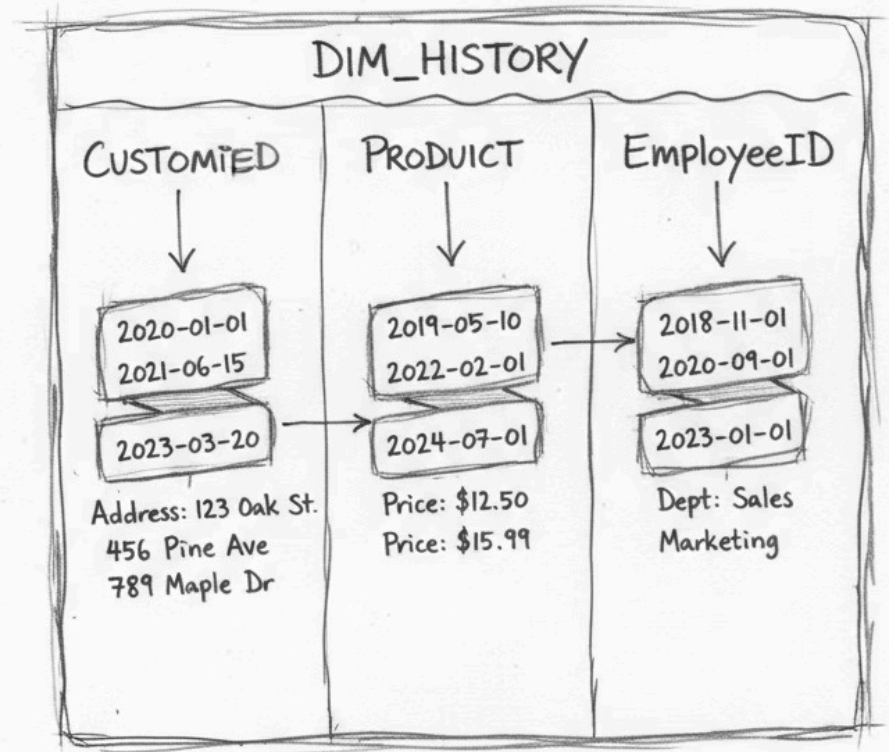
Avoiding logic duplication

If Gold needs the same transformation as Silver, that means Silver is incomplete — not that Gold should reimplement it.

Slowly Changing Dimensions

Slowly Changing Dimensions (SCD) is a strategy for handling changes in Data Warehouse dimension tables. Customer addresses change, products change price, employees move to other departments – how do you preserve the history of these changes?

Different SCD types represent various trade-offs between storage cost, query complexity, and business requirements for history. The most popular in practice are Type 1 (overwriting) and Type 2 (full history).



SCD Type 1 – Overwrite

Strategy: Overwrite old value

SCD Type 1 is the simplest approach – when an attribute changes, we overwrite the old value with the new one. **History is lost**. Used for attributes that do not require auditing (e.g., error correction, insignificant changes).

```
MERGE INTO dim_customer AS target
USING customer_updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
  UPDATE SET
    target.address = source.address,
    target.phone = source.phone,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT *;
```

Transformation example

```
-- State before update
customer_id | address | phone
1001       | Warsaw | 123-456
```

```
-- New data
customer_id | address | phone
1001       | Krakow  | 789-012
```

```
-- State after MERGE (Type 1)
customer_id | address | phone
1001       | Krakow  | 789-012
```

✓ **Advantages:** Simplicity, low storage

✗ **Disadvantages:** No history, no audit

SCD Type 2 – Full History

SCD Type 2 preserves the full history of changes by creating a new row for each version of a record. Key columns: `is_current`, `effective_start_date`, `effective_end_date`. It enables point-in-time queries.

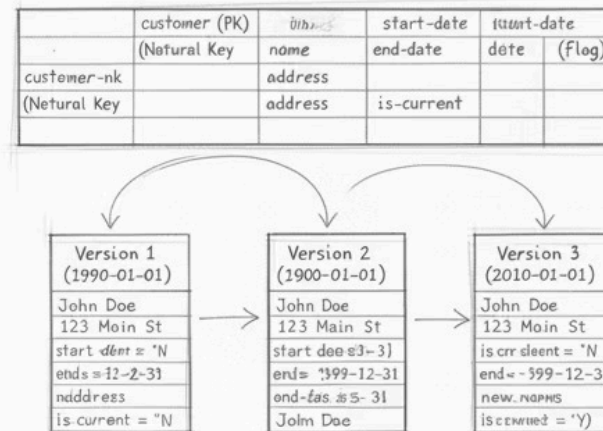
```
MERGE INTO dim_product_scd2 AS target
USING (
  SELECT *, current_date() as effective_date FROM product_updates
) AS source
ON target.product_id = source.product_id
AND target.is_current = true
```

```
-- Close out current record
WHEN MATCHED AND (
  target.price != source.price OR
  target.category != source.category
) THEN UPDATE SET
  target.is_current = false,
  target.effective_end_date = source.effective_date
```

```
-- Insert new version
WHEN NOT MATCHED THEN INSERT (
  product_id, name, price, category,
  is_current, effective_start_date, effective_end_date
) VALUES (
  source.product_id, source.name, source.price, source.category,
  true, source.effective_date, '9999-12-31'
);
```

```
-- Insert new current version for updated records
INSERT INTO dim_product_scd2
SELECT * FROM source WHERE exists_in_target_as_outdated;
```

CUSTOMER_DIM (Type 2 SCD)



SQL Code: Update & Insert

```
- 1. Close out CUSTOMER_DIM
UPDATE out enddd record
SET end-date = CURRENTDATE - 1,
WHERE customer nk = new_customer nk
AND is-current = 'Y',
)

- 2. Insert new record
INSERT INTO CUSTOMER_DIM (
  customer sk, customer nk, name,
  address, start new_name = customer nk
) VALUES (next surrogate key, new_address )
CURRENTDATE, '9999-12-31' 'Y')
)
```

SCD Type 2 – History Example

product_id	name	price	is_current	effective_start	effective_end
P001	Laptop X	4500	false	2023-01-01	2023-06-30
P001	Laptop X	4200	false	2023-07-01	2024-01-15
P001	Laptop X Pro	4200	true	2024-01-16	9999-12-31

-- Point-in-time query: what was the price in Q3 2023?

```
SELECT price
FROM dim_product_scd2
WHERE product_id = 'P001'
AND '2023-09-15' BETWEEN effective_start_date AND effective_end_date;
```

-- Result: 4200

-- Current version

```
SELECT * FROM dim_product_scd2 WHERE product_id = 'P001' AND is_current = true;
```

SCD Type 3 & 4 – Alternative Strategies

SCD Type 3 – Previous Value

Storing only one previous attribute value in a dedicated column (e.g., `previous_address`). Limited history – only *current* and *previous*.

id	address	prev_address
1001	Krakow	Warsaw

Usage: "Before/after" analysis, tracking recent changes, low storage overhead.

SCD Type 4 – History in a Separate Table

Splitting into two tables: `dim_customer` (current state) + `dim_customer_history` (full audit trail). Separation of hot data from cold data.

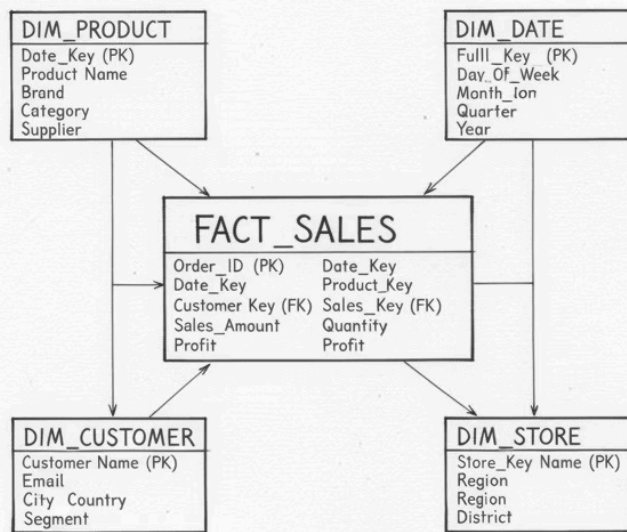
- `dim_customer`: current records only
- `dim_customer_history`: all historical versions

Usage: Optimizing query performance on current state, compliance requirements for long-term audit.

Type 3 and 4 are less commonly used in practice – most cases involve a choice between Type 1 (simple overwrite) vs Type 2 (full history).

Gold Layer – Business-Ready Data

The Gold Layer is a **business consumption layer** – data is aggregated, denormalized, and optimized for specific use cases (BI dashboards, ML features, API endpoints). Gold implements classic Data Warehouse patterns: Star Schema, Fact & Dimension tables.



Fact Tables

Transactions, events, measurements.
High cardinality, frequent inserts.
Contain foreign keys to dimensions
and metrics (amount, quantity).

Dimension Tables

Business entities (customers, products, locations). Low cardinality, infrequent updates. Contain descriptive attributes for slice & dice.

Aggregates

Pre-computed summaries (daily_sales, monthly_kpis). Drastic reduction in scan cost for typical queries. Materialized views.

GOLD LAYER

Gold as a Product Layer — Not Generic

The most common mistake in designing the Gold layer: treating it like "just another Silver transformation." Gold is a **data product** — designed for a specific use case, optimized for a specific consumer.

BI and Reporting

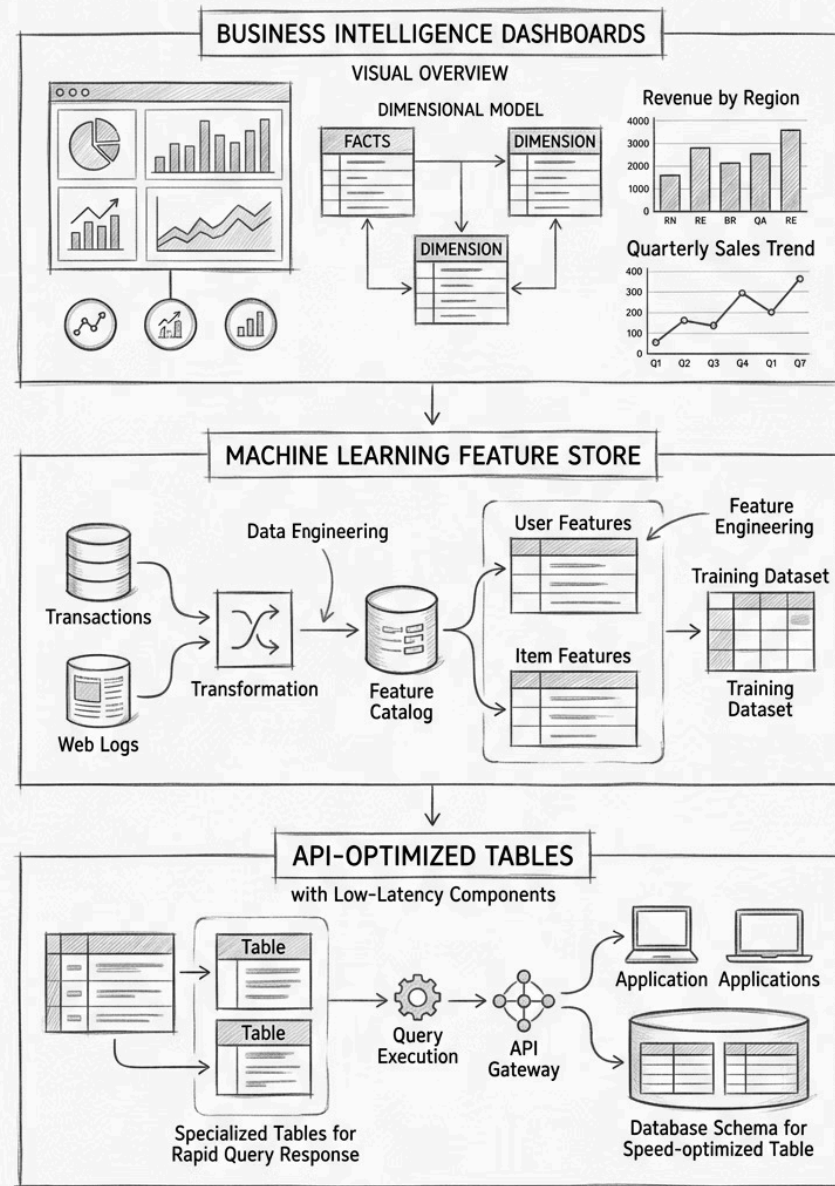
Dimensional models, aggregates, pre-joined fact tables. Optimized for Power BI / Tableau queries — proper partitioning and Z-ORDER.

Machine Learning

Feature store, training sets, validation data. Format and granularity tailored to model requirements — not reporting.

API and Applications

Tables optimized for point lookups, low latency, predictable schemas. Often materialized views or Delta with liquid clustering.



Managing Dependencies Between Layers

In complex data platforms, dependencies between tables form a directed acyclic graph (DAG). Lack of awareness of this structure leads to hidden dependencies, cascading failures, and an inability to determine the impact of schema changes.

Clear dependency structure

- Each Silver table knows its source in Bronze — and only that source
- Gold tables read from Silver, never directly from Bronze
- Changes propagate in one direction: Bronze → Silver → Gold
- Use lineage in Unity Catalog to document and visualize the graph

Hidden dependencies — warning signs

- A Gold notebook reading directly from Bronze
- A Silver transformation hardcoded in Workflow, not in code
- A table used by several pipelines without a shared owner
- Lack of documentation about which task produces a given table

DEMO

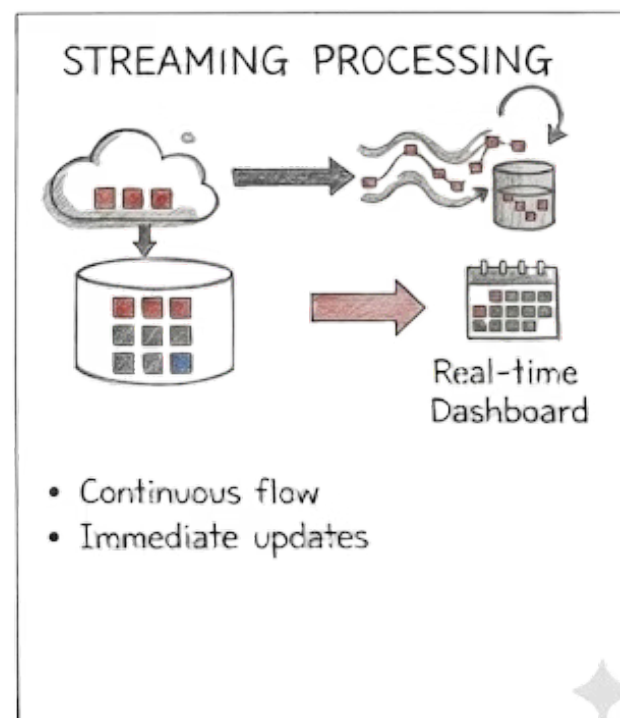
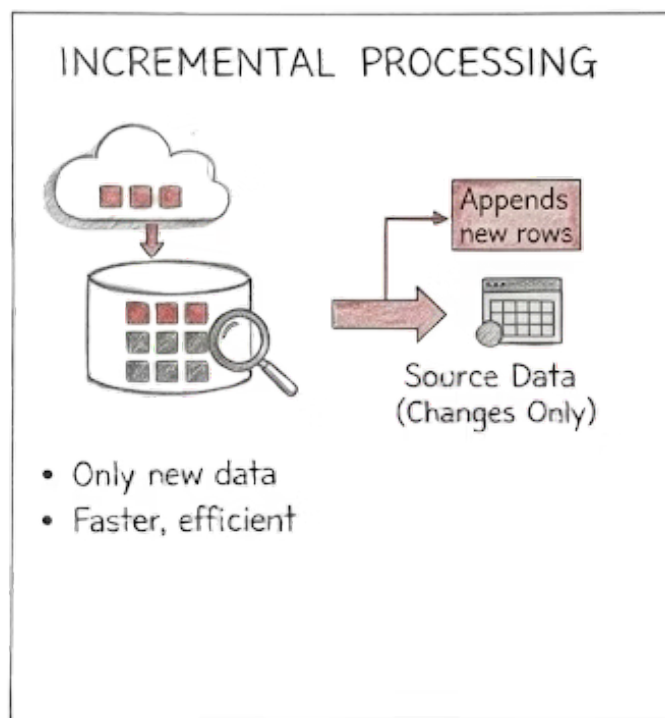
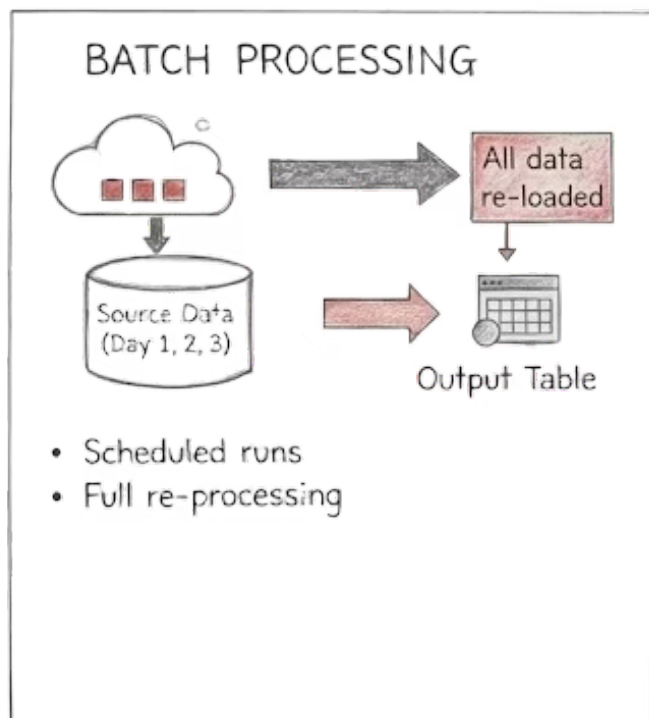
Let's see Databricks in action!

Data Ingestion

Loading data into Lakehouse

Data Processing Patterns

DATA PROCESSING PATTERNS



Batch and Streaming — One System, Not Two

A common architectural mistake: maintaining two separate pipelines — one batch and one streaming — for the same data. This doubles maintenance cost and leads to divergent results. Unified processing is an approach where the same logic handles both modes.

Streaming — for incremental updates

Processing new data immediately after it appears. Ideal for operational data requiring low latency: events, logs, real-time transactions. In DLT: streaming tables with Auto Loader as the source.

Batch — for historical loading

Full or incremental processing of large historical volumes. Ideal for backfills, recalculations after business rule changes, and daily reporting. In DLT: standard tables with full refresh or **FULL REFRESH**.

i Key principle: the transformation logic should be identical for both modes. The difference lies only in the triggering method and schedule — not in the business code.

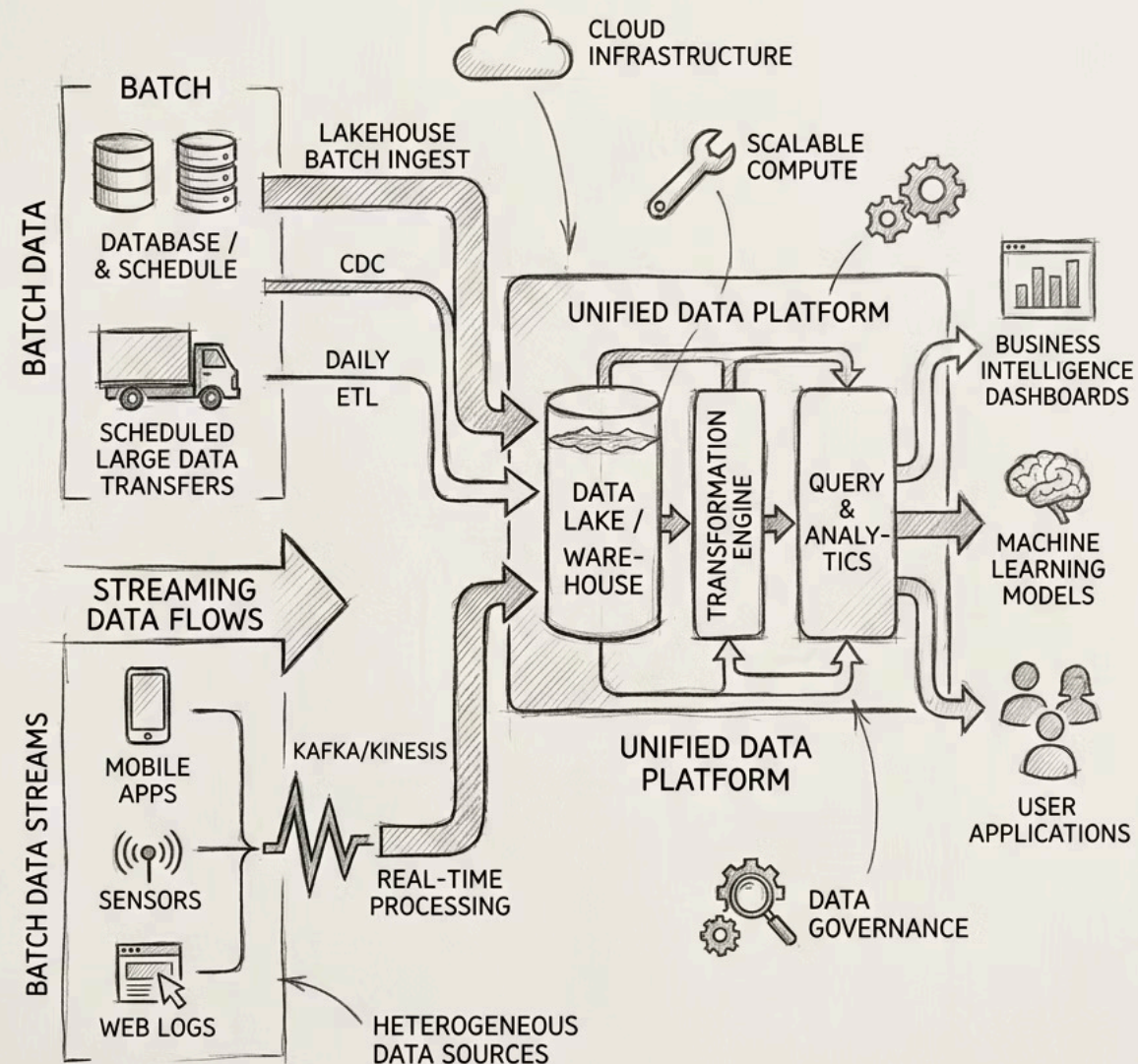
Batch vs Streaming Comparison

Feature	Batch (COPY INTO)	Streaming (Auto Loader)
Latency	Minutes – Hours	Seconds – Minutes
File size	Large files >1GB	Small files <100MB
Frequency	Scheduled (hourly/daily)	Continuous / Near real-time
Compute cost	Lower (on-demand)	Higher (always-on clusters)
Key feature	Idempotency, File tracking	Schema evolution, Checkpoints
Use case	Daily exports, Historical loads	IoT sensors, Log aggregation, CDC
State management	Simple (file tracking)	Complex (checkpoint management)

Hybrid Pipeline Pattern

Most production systems require a hybrid approach: a one-time historical data load (batch) plus continuous incremental updates (streaming). The key is for both modes to write to the same target tables with the same processing logic.

- ❑ In Delta Lake, both paths can write to the same table thanks to ACID transactions. You do not need separate tables for historical and incremental data — this is a common anti-pattern that adds complexity without value.



Databricks Data Loading Methods

Databricks offers three main methods for loading data into a Delta Lake environment, each optimized for different scenarios:

CTAS (CREATE TABLE AS SELECT)

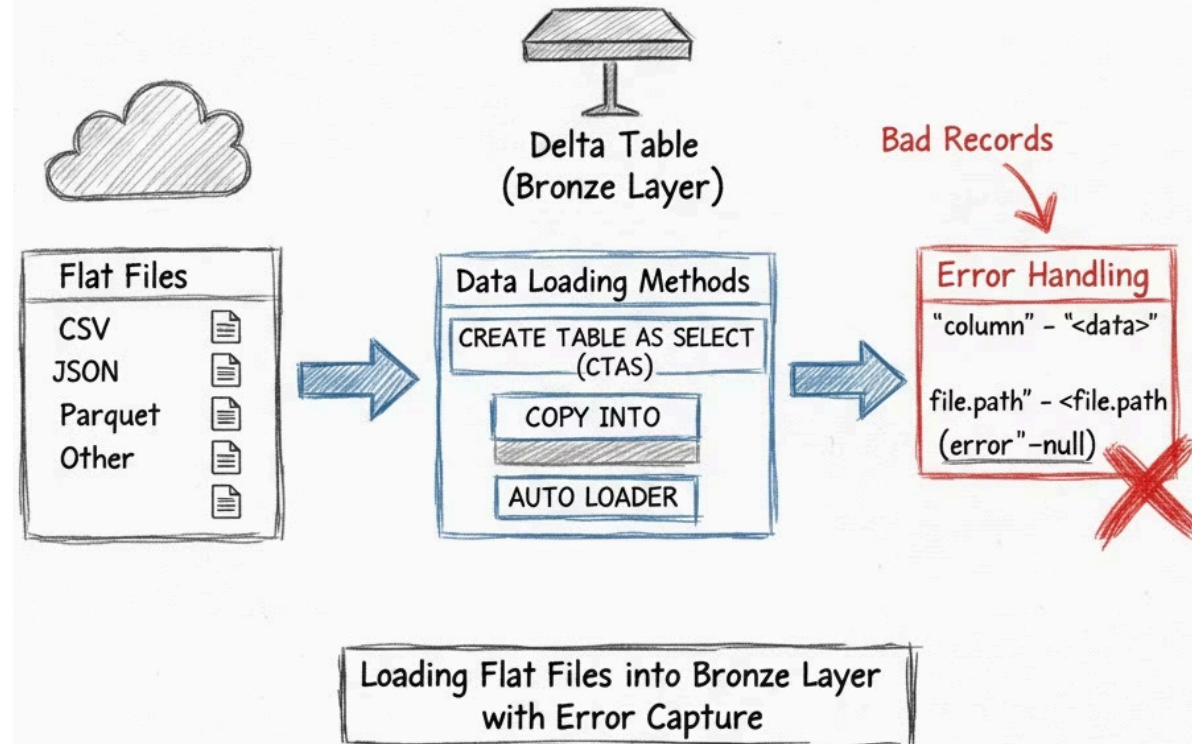
This method creates a new Delta Lake table based on the results of a SQL query. It is ideal for one-time table creation, data replication, or transforming existing data where the full dataset is already available.

COPY INTO

COPY INTO is an efficient tool for batch loading files from external locations (e.g., S3, ADLS) into Delta Lake tables. It supports idempotency, which prevents data duplication, and is designed for regular loading of large batches of data from files.

Auto Loader

Auto Loader is a streaming data ingestion mechanism that automatically and incrementally processes new data files as soon as they appear in the source. It is an optimal solution for continuous data processing, streaming scenarios, and ensuring real-time data freshness.



CTAS – CREATE TABLE AS SELECT

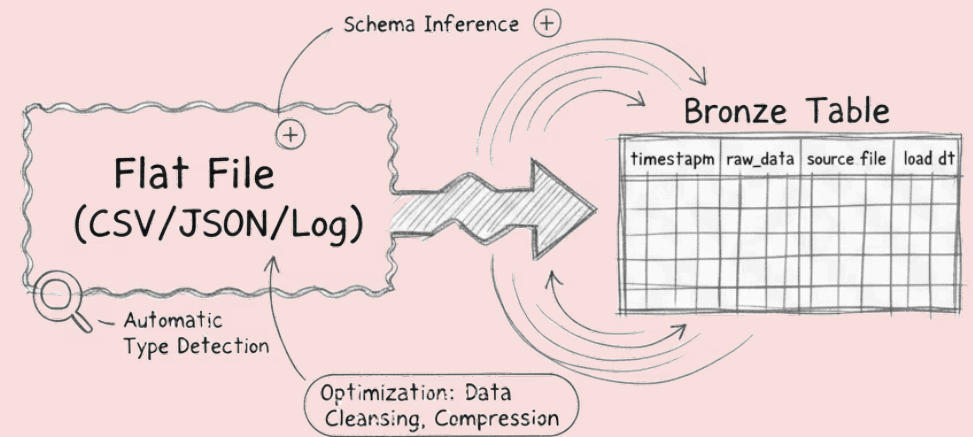
```
-- CTAS with automatic schema inference
CREATE OR REPLACE TABLE raw_products
AS
SELECT * FROM read_files(
  '/Volumes/landing/products.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

-- - Table structure
DESCRIBE TABLE raw_products;
```

CTAS (CREATE TABLE AS SELECT) is a versatile method for quickly creating new tables, ideal for initial data loading from external sources such as CSV files. It enables automatic detection of the source file schema (schema inference) and the application of basic data transformations (e.g., column renaming, type conversion) directly during table creation.

Advantages of Schema Inference in CTAS:

- Automatic data type detection
- No need to define schema upfront
- Rapid prototyping
- Support for various formats (CSV, JSON, Parquet)



**CREATE TABLE bronze_table AS
SELECT * FROM flat-file_source WHERE...**

COPY INTO – Batch Ingestion

```
COPY INTO bronze_sales
FROM 's3://bucket/sales/year=2024/'
FILEFORMAT = PARQUET
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');

-- Automatic file tracking
SELECT * FROM bronze_sales;
```

The `COPY INTO` command is the simplest and most effective way to load batch data into Delta Lake. A key feature is **idempotency** – running it multiple times on the same set of files will not cause duplicates.

When to use COPY INTO

- Files >1GB (optimal size for Parquet)
- Schedules every few hours or less frequently
- Initial bulk loads of historical data
- Exports from external systems (daily dumps)
- Low latency requirements (SLA >15 min)

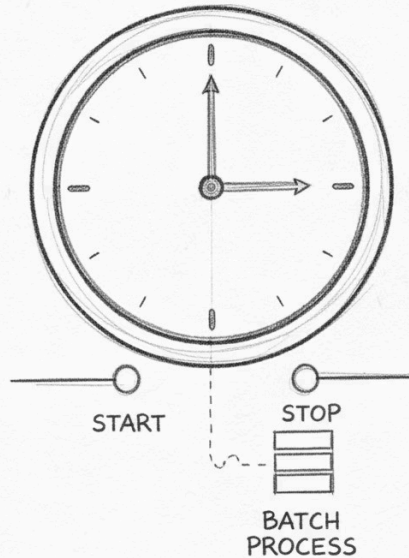
 `COPY INTO` tracks loaded files in the metastore – a file loaded once will not be loaded again.

Trigger Modes – Controlling Latency and Costs

Trigger mode determines the frequency of micro-batch execution in a streaming pipeline. This is a key decision influencing the trade-off between latency and compute costs.

Trigger Mode	Description	When to Use	Cost
<code>availableNow=True</code>	Processes all available data and stops	Scheduled jobs, cost optimization, batch-like streaming	Low
<code>processingTime="10second"</code>	Micro-batch every 10 seconds (continuous)	Real-time dashboards, operational analytics	High
<code>once=True</code>	Single micro-batch and exit	Testing, one-off processing	Very Low
Default (no trigger)	Continuous processing ASAP	Ultra-low latency (<1s), fraud detection	Very High

availableNow – Recommended Trigger Mode



```
(df.writeStream
  .format("delta")
  .trigger(availableNow=True)
  .option("checkpointLocation", checkpoint_path)
  .table("bronze_table"))
```

`availableNow=True` is the best of both worlds: streaming semantics (exactly-once, checkpoint management, schema evolution) with the economics of batch processing.

Advantages of availableNow

- Cluster only starts when new data is available
- Full control over scheduling via Workflows
- Identical logic for batch and streaming
- Compute cost reduction of 70–90% vs continuous

processingTime – Real-time trigger

```
(df.writeStream
  .format("delta")
  .trigger(processingTime="10 seconds")
  .option("checkpointLocation", checkpoint_path)
  .table("bronze_table"))
```

The cluster remains active and runs a micro-batch every 10 seconds

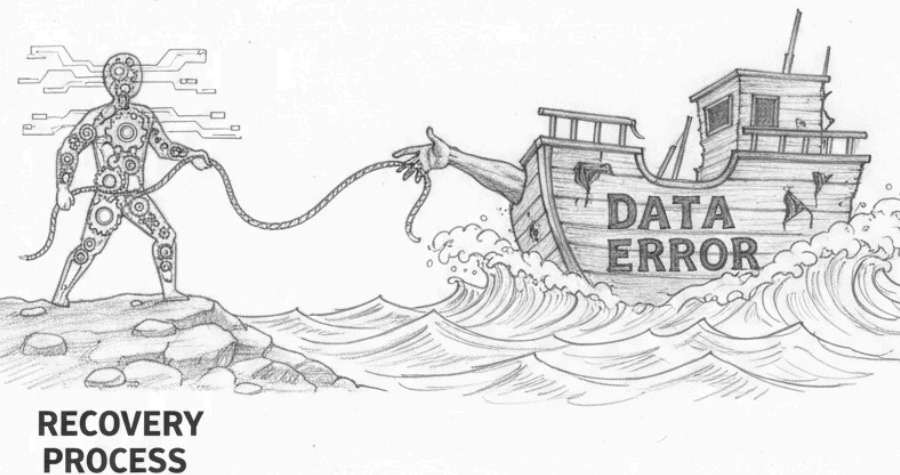
The `processingTime` trigger runs a micro-batch at fixed intervals, regardless of data availability. The cluster must remain active non-stop, which generates high compute costs.

✓ When to use

- Real-time operational dashboards
- SLA for latency <1 minute
- Fraud detection / anomaly detection
- Live monitoring of IoT systems

⚠ Limitations

- Cluster always-on = high DBU costs
- Empty micro-batch if no data
- More difficult lifecycle management
- Requires dedicated cluster pools



Rescued Data Column – Error Handling

Auto Loader automatically adds a `\_rescued\_data` column to every Bronze table. It contains raw JSON from records that do not match the expected schema – instead of rejecting bad data, Delta Lake preserves it for later analysis.

-- Check for problematic records

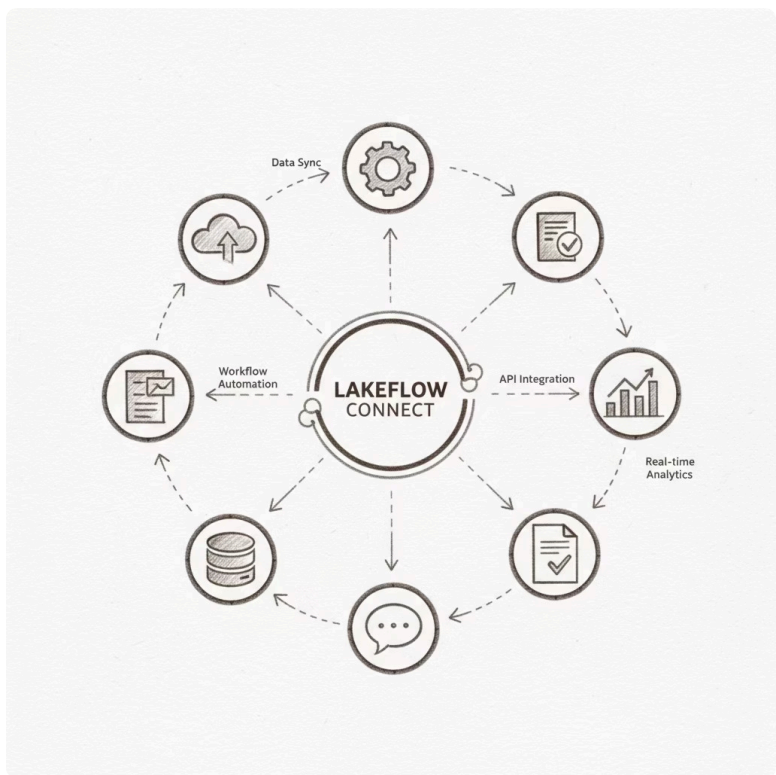
```
SELECT _rescued_data, count(*) as error_count
FROM bronze_events
WHERE _rescued_data IS NOT NULL
GROUP BY _rescued_data
ORDER BY error_count DESC;
```

-- Typical rescued record

```
{"unexpected_field": "value", "malformed_json": "..."}"
```

This is a key data quality mechanism – instead of silent failure, we gain visibility into data source issues and the ability to retrospectively fix them.

Lakeflow Connect – SaaS Integration



Connector	Use Case
Salesforce	CRM data, Opportunities, Accounts
SAP	ERP data, Financial transactions
Workday	HR data, Employee records
ServiceNow	IT Service Management, Incidents
Google Analytics	Web analytics, User behavior
HubSpot	Marketing automation, Leads

Lakeflow Connect is a zero-code solution for integrating with popular SaaS systems. Automatic Auth configuration, Schema Discovery, Incremental Load, and Error Handling.

Lakeflow Connect eliminates the need to build custom connectors and manage API rate limits, pagination, authentication refresh, etc. Everything is managed by Databricks.

DEMO

Let's see Databricks in action!

Lakeflow Pipelines

Modern ETL

What are Lakeflow Pipelines?

Lakeflow (formerly Delta Live Tables) is a framework for **declaratively building data pipelines** in Databricks. Instead of writing orchestration code, we define the **final outcome** – the framework manages execution, monitoring, and data quality.

Traditional Approach

- Manual state management
- Custom retry logic
- Custom checkpoints
- Manual dependency tracking
- Separate quality control tools

Lakeflow Pipelines

- Declarative table definitions
- Automatic retries and error handling
- Built-in checkpointing
- Automatic lineage graph
- Built-in Expectations (quality rules)

Where Does Lakeflow Fit in the Architecture?

Lakeflow (Delta Live Tables) is a **transformation engine and dependency manager** — not a complete architecture. Understanding its role and limitations is key to using it correctly.

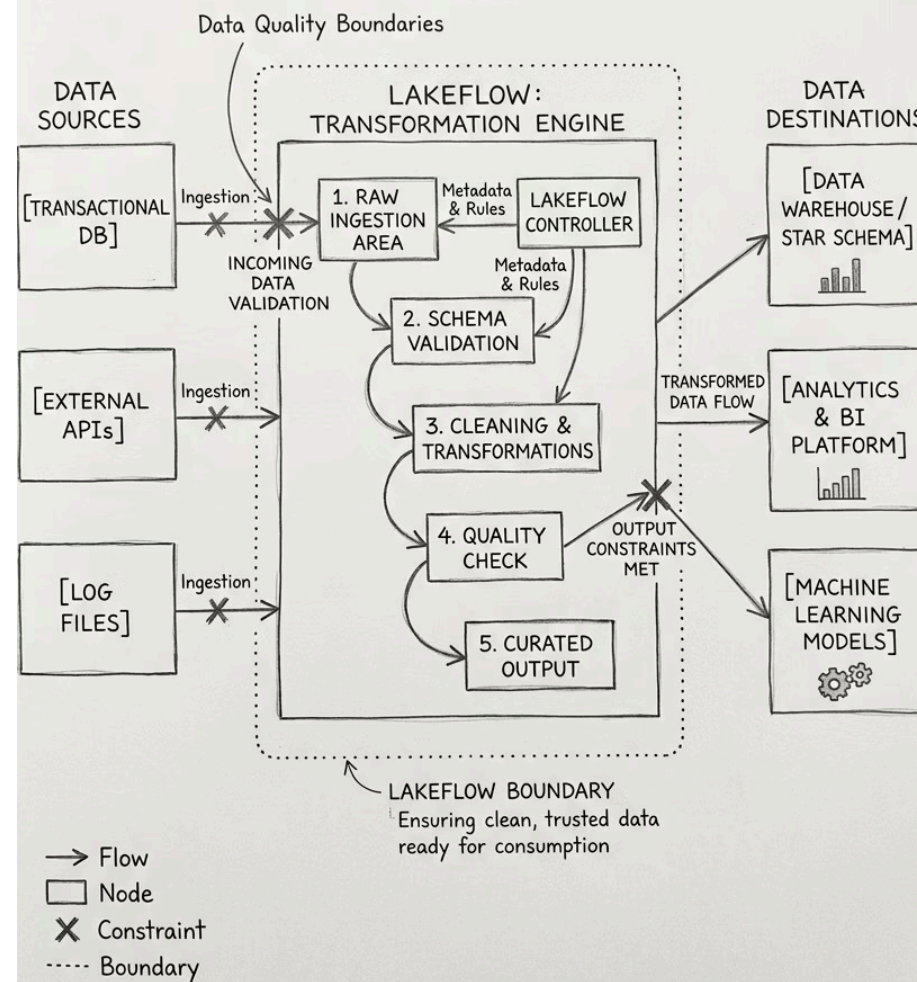
What Lakeflow does

- Declarative definition of tables and their dependencies
- Automatic management of processing order
- Support for incremental processing (Change Data Feed)
- Built-in data quality constraints (EXPECT)

What Lakeflow does not do

- Does not replace the orchestration layer (Workflows)
- Does not define the Medallion architecture — that is your responsibility
- Does not manage scheduling and retries at the system level
- Does not handle dependencies between different DLT pipelines

DATA ARCHITECTURE WITH LAKEFLOW



What happened to Delta Live Tables (DLT)?

Delta Live Tables (DLT) has been updated to **Lakeflow Spark Declarative Pipelines (SDP)**. Existing DLT code is fully compatible, but it is recommended to update to the new API names for future features and compatibility with Apache Spark™ Declarative Pipelines (from Spark 4.1).

Key changes in the Python API:

~~@dlt~~ → @dp

Replacing the library reference.

~~@table (general)~~ → @table
(streaming) /
@materialized_view

A dedicated decorator for materialized views has been introduced, increasing readability.

~~@view~~ → @temporary_view

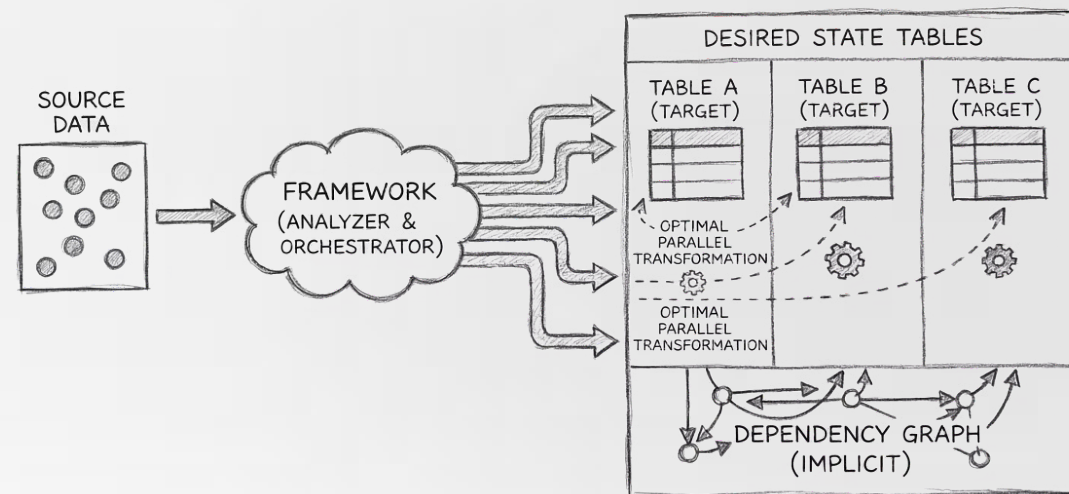
Clear distinction for temporary views.

 **Note:** You may still encounter references to the DLT name in some places in Databricks (e.g., classic SKUs, event log schemas). The existing Python API with DLT names is still supported, but transitioning to the new naming convention is recommended.

Declarative Approach – "WHAT" instead of "HOW"

In the declarative approach, we describe the desired state of the data (what we want to achieve), instead of imperative steps (how to do it). Lakeflow independently analyzes dependencies and optimizes the execution of transformations.

DECLARATIVE DATA PIPELINE FRAMEWORK



What Makes Up Lakeflow Pipelines?

Lakeflow Pipelines is a framework integrating key aspects of data engineering, enabling the building of reliable and efficient pipelines based on the Lakehouse paradigm.



Declarative Definitions

Focus on "**WHAT**" you want to achieve with data, not "**HOW**" to implement it. Lakeflow automatically manages execution logic.



Automatic Orchestration

The framework independently manages task order, dependencies, retries in case of errors, and progress monitoring.



Built-in Data Quality

Define data quality rules directly in the pipeline code, ensuring automatic validation and control over data.



Streaming and Materialized Views

A unified approach to processing streaming and batch data within a single pipeline.



Simplified Compute

Serverless options minimize the need for infrastructure management, automatically scaling resources based on load.

CREATE OR REFRESH Syntax

Streaming Table

For append-only sources (logs, events, transactions):

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT *, current_timestamp() as ingestion_time
FROM read_files(
  "/Volumes/raw/orders/", format=>"csv", header=>true,
  inferSchema=>true
)
```

A streaming table automatically processes new files.

Materialized View

For full refresh or incremental update transformations:

```
CREATE OR REFRESH MATERIALIZED VIEW silver_orders
AS SELECT
  order_id, customer_id, order_date, total_amount, items
FROM STREAM(bronze_orders)
WHERE order_status != 'cancelled'
```

`STREAM()` indicates incremental processing.

When to use Streaming vs. Materialized?

Streaming Table

- Data is append-only (events, logs)
- Exactly-once semantics required
- Data arrives in real-time
- Bronze/Raw Layer – retaining all data
- CDC from transactional databases

Examples: application logs, Kafka streams, CDC from PostgreSQL

Materialized View

- UPDATE/DELETE required (SCD2, deduplication)
- Aggregations and complex joins
- Gold Layer – business metrics
- Snapshot Tables – current state only
- Feature Tables for ML

Examples: Calculating CLV, daily active users, product recommendation features

Data Flows in Lakeflow Pipelines

A flow is a combination of a query and a target table, processing data in batch or streaming mode, incrementally in a Lakeflow Spark Declarative Pipelines pipeline.

Definition and Types of Flows

- Flows are created **automatically** when defining a table or view (e.g., CREATE OR REFRESH STREAMING TABLE).
- They can be defined **explicitly** (CREATE FLOW / @dp.append_flow) for complex scenarios.
- Standard flows (append flows) **add new rows**, ideal for data ingestion and transformation.
- **Auto CDC** flows are used to ingest data with changes (Change Data Capture).

Applications and Benefits

- Every pipeline update triggers a flow that refreshes the tables.
- This can be an **incremental refresh** (only new records) or a **full refresh**.
- Enables **combining data from multiple sources** into one target table (e.g., adding new regions) without a full refresh.
- Allows **backfilling historical data** using the INSERT INTO ONCE syntax.

Multiple Flows to a Single Target

Creating a table and assigning a flow

You can first create a streaming table and then define a flow to it. The example below shows how to independently create a table and assign a single flow, achieving the same results as with default flow creation.

```
-- utwórz tabelę strumieniową  
CREATE OR REFRESH STREAMING TABLE customers_silver;  
  
-- dodaj przepływ do tabeli customers_silver  
CREATE FLOW customers_silver_flow  
AS INSERT INTO customers_silver BY NAME  
SELECT * FROM STREAM(customers_bronze);
```

Combining data from multiple sources

Using multiple append flows (append_flow in Python or CREATE FLOW...INSERT INTO in SQL) is ideal for combining data from different sources into a single streaming table. This ensures incremental updates, which are more efficient than using a UNION clause.

```
-- utwórz tabelę strumieniową  
CREATE OR REFRESH STREAMING TABLE customers_us;  
  
-- dodaj pierwszy przepływ dopisujący z danych z zachodniego USA  
CREATE FLOW append_us_west  
AS INSERT INTO customers_us BY NAME  
SELECT * FROM STREAM(customers_us_west);  
  
-- dodaj drugi przepływ dopisujący z danych ze wschodniego USA  
CREATE FLOW append_us_east  
AS INSERT INTO customers_us BY NAME  
SELECT * FROM STREAM(customers_us_east);
```

❏ **Important:** Data quality expectations should be defined on the **target table** (e.g., customers_us), not in the @dp.append_flow flow definition.

Flows are identified by name, used to manage streaming checkpoints. Renaming an existing flow is treated as creating a new one, resetting its checkpoints. Additionally, the same flow name cannot be reused within a single pipeline.

Backfilling Historical Data

Lakeflow Pipelines makes it easy to backfill historical data in existing pipelines. This is crucial when you need to process data from periods not included in the initial pipeline configuration.

Defining the Scope

Create a new flow, for example, by removing or changing `modifiedAfter`.

One-time run

Use the `INSERT INTO ONCE` syntax for a one-time execution and to avoid duplicates.

Integration

New historical data will be seamlessly integrated into the target table.

```
-- create the streaming table
CREATE OR REFRESH STREAMING TABLE registration_events_raw;

-- original incremental flow (e.g., from 2025 onwards)
CREATE FLOW registration_events_raw_incremental
AS INSERT INTO registration_events_raw BY NAME
SELECT * FROM STREAM read_files(
  "/Volumes/gc/demo/apps_raw/event_registration/*",
  format => "json",
  modifiedAfter => "2024-12-31T23:59:59.999+00:00"
);

-- one-time backfill for 2024
CREATE FLOW registration_events_raw_backfill_2024
AS INSERT INTO ONCE registration_events_raw BY NAME
SELECT * FROM read_files(
  "/Volumes/gc/demo/apps_raw/event_registration/year=2024/*",
  format => "json"
);
```

CDC APPLY in Lakeflow Pipelines (SCD Type 1 and 2)

What is AUTO CDC INTO?

AUTO CDC INTO automates data change management, replacing manual MERGE INTO logic. It simplifies data integration from transactional systems where tracking record evolution is crucial.

KEYS (column)

Specifies one or more columns that form a unique record key.

APPLY AS DELETE WHEN ...

Defines a logical condition for when a source record should be treated as a deletion.

APPLY AS TRUNCATE WHEN ...

Defines a condition that causes all records in the target table to be deleted before applying changes.

SEQUENCE BY column

Used to handle out-of-order events by specifying the latest change.

COLUMNS * EXCEPT (columns)

Indicates all source columns to include, except for those listed.

STORED AS SCD TYPE 1

Changes update records, overwriting old values (only the latest version).

STORED AS SCD TYPE 2

Changes are stored as new records with `__START_AT` and `__END_AT` columns to track history.

SCD Type 1: Retaining the Latest Data

In SCD Type 1, data updates overwrite existing records in the target table, always ensuring the latest version of the data. Lakeflow Pipelines automatically manages this logic.

```
CREATE OR REFRESH STREAMING TABLE target;

CREATE FLOW flowname AS AUTO CDC INTO target
FROM stream(cdc_data.users)
KEYS (userId)
APPLY AS DELETE WHEN operation = "DELETE"
APPLY AS TRUNCATE WHEN operation = "TRUNCATE"
SEQUENCE BY sequenceNum
COLUMNS * EXCEPT (operation, sequenceNum)
STORED AS SCD TYPE 1;
```

After applying SCD Type 1 updates, the target table contains the latest records:

userId	name	city
124	Raul	Oaxaca
125	Mercedes	Guadalajara
126	Lily	Cancun

SCD Type 2: Tracking Historical Changes

SCD Type 2 tracks all changes, preserving historical versions of records. Each change creates a new record with a new validity period (`__START_AT` and `__END_AT`), which is crucial for analyzing changes over time.

```
CREATE OR REFRESH STREAMING TABLE target;

CREATE FLOW target_flow AS AUTO CDC INTO target
FROM stream(cdc_data.users)
KEYS (userId)
APPLY AS DELETE WHEN operation = "DELETE"
SEQUENCE BY sequenceNum
COLUMNS * EXCEPT (operation, sequenceNum)
STORED AS SCD TYPE 2;
```

After applying SCD Type 2 updates, the target table contains historical records:

userId	name	city	__START_AT	__END_AT
123	Isabel	Monterrey	1	5
123	Isabel	Chihuahua	5	6

Automated Maintenance Tasks

Lakeflow Pipelines autonomously performs key maintenance tasks on managed tables, utilizing advanced predictive optimization. This ensures better query performance and cost reduction by removing stale data and organizing it.

Key Operations

The pipelines perform **OPTIMIZE** (file compaction) and **VACUUM** (removal of old data) operations, ensuring optimal storage utilization.

Intelligent Scheduling

Maintenance tasks are scheduled by the predictive optimization mechanism and run only after a pipeline update.

Benefits

Automatic maintenance keeps tables in optimal condition, resulting in [increased query performance](#), [reduced resource consumption](#), and [lower storage costs](#).

📄 To learn more about the frequency of predictive optimization and associated costs, refer to the **predictive optimization system table**.

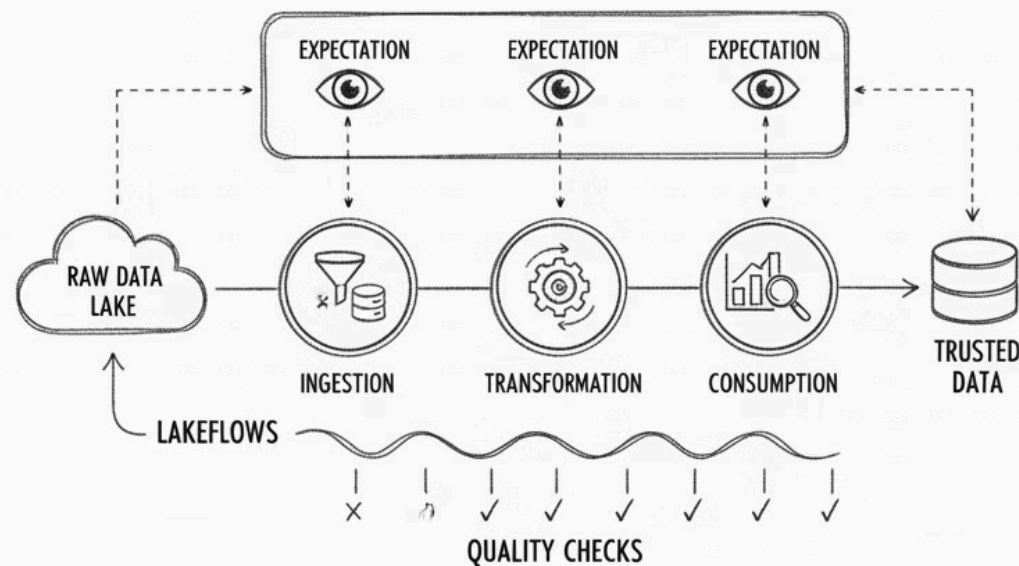
Expectations

In the context of Lakeflow Pipelines, "Expectations" define data quality rules and tests that are automatically verified during processing. They ensure data reliability and early anomaly detection.

Data Quality – Expectation

Expectations are declarative data quality rules, built directly into the pipeline. They act like unit tests for data, automatically verifying conditions with each processing. They block the propagation of erroneous data to downstream tables.

Expectations in Lakeflows



Types of Violation Reactions

DELETE ROW

Rows that do not meet the condition are automatically skipped. Use when erroneous data can be safely ignored.

```
CONSTRAINT valid_email  
EXPECT (email LIKE '%@%')  
ON VIOLATION DROP ROW
```

CANCEL UPDATE

The entire batch is rejected if any row violates the condition. Apply to critical business rules.

```
CONSTRAINT positive_amount  
EXPECT (amount > 0)  
ON VIOLATION FAIL UPDATE
```

TRACK ONLY (DEFAULT)

The violation is recorded in metrics, but data continues to flow. Ideal for monitoring quality without blocking the pipeline.

```
CONSTRAINT recent_data  
EXPECT (  
    event_date >= current_date() - 7  
)
```

Metrics: All violations are automatically recorded in the pipeline event log and are available in the user interface as metrics for each table.

Best Practices for Expectations

Why is this important?

Good practices facilitate the application, updating, and auditing of expectations rules across multiple datasets and pipelines, without modifying code. They increase validation consistency, reduce maintenance costs, and improve solution portability.

Definition Separation

Store expectation definitions separately from pipeline logic

Tagging

Add custom tags to group related expectations

Reusability

Apply the same expectations across multiple pipelines

Centralized Expectations Repository: Delta Table

The `rules` table centralizes validation rules, storing the name, SQL constraint, and a tag for filtering.

```
CREATE OR REPLACE TABLE rules AS
SELECT col1 AS name, col2 AS constraint, col3 AS tag
FROM (
  VALUES
    ("website_not_null",
     "Website IS NOT NULL",
     "validity"),
    ("fresh_data",
     "to_date(updateTime,'M/d/yyyy h:m:s a') > '2010-01-01'",
     "maintained"),
    ("social_media_access",
     "NOT(Facebook IS NULL AND Twitter IS NULL AND Youtube IS NULL)",
     "maintained")
)
```

It simplifies rule management via SQL, allowing easy addition of new constraints and categorization by business tags.

Applying rules in a Python pipeline

The `get_rules()` function retrieves validation rules from the table, matching them by tag. The `@dp.expect_all_or_drop()` decorator automatically removes records that do not meet these criteria.

```
from pyspark import pipelines as dp
from pyspark.sql.functions import expr, col

def get_rules(tag):
    df = spark.read.table("rules").filter(col("tag") == tag).collect()
    return {row['name']: row['constraint'] for row in df}

@dp.table
@dp.expect_all_or_drop(get_rules('validity'))
def raw_farmers_market():
    return (
        spark.read.format('csv').option("header", "true")
        .load('/databricks-datasets/data.gov/farmers_markets_geographic_data/')
    )

@dp.table
@dp.expect_all_or_drop(get_rules('maintained'))
def organic_farmers_market():
    return spark.read.table("raw_farmers_market").filter(expr("Organic = 'Y'"))
```

Schema Evolution

This pattern enables the migration of data sources and the management of multiple versions, ensuring backward compatibility and data quality.

```
CREATE OR REFRESH MATERIALIZED VIEW evolving_table(  
  CONSTRAINT valid_migrated_data EXPECT (  
    (col1 IS NOT NULL AND col2 IS NOT NULL) AND  
    (CASE WHEN col3 IS NOT NULL THEN col3 > 0 ELSE TRUE END)  
  ) ON VIOLATION FAIL UPDATE  
) AS  
SELECT * FROM new_source  
UNION  
SELECT *, NULL as col3 FROM legacy_source;
```

UNION combines new and old sources. CASE handles optional columns. ON VIOLATION FAIL UPDATE stops the pipeline on critical errors.

Row Count Verification

Monitoring the number of rows in materialized views using COUNT(*) allows for quick record verification and anomaly identification, helping to maintain data quality.

```
CREATE OR REFRESH MATERIALIZED VIEW  
count_verification(  
  CONSTRAINT no_rows_dropped  
  EXPECT (a_count == b_count)  
) AS  
SELECT * FROM  
  (SELECT COUNT(*) AS a_count FROM table_a),  
  (SELECT COUNT(*) AS b_count FROM table_b)
```

The constraint EXPECT (a_count == b_count) automatically detects discrepancies in row counts between source tables.

Detecting Missing Records and Key Uniqueness

Missing Record Detection

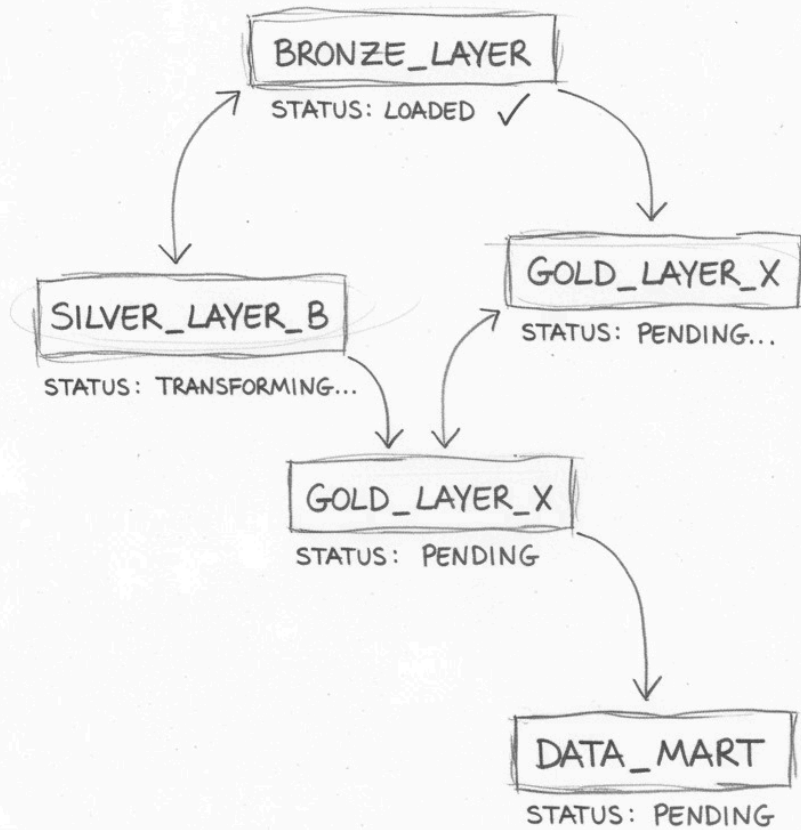
```
CREATE OR REFRESH MATERIALIZED VIEW  
report_compare_tests(  
  CONSTRAINT no_missing_records  
  EXPECT (r_key IS NOT NULL)  
) AS  
SELECT v.*, r.key as r_key  
FROM validation_copy v  
LEFT OUTER JOIN report r  
ON v.key = r.key
```

LEFT JOIN reveals records present in validation but missing in the report.

Primary Key Validation

```
CREATE OR REFRESH MATERIALIZED VIEW  
report_pk_tests(  
  CONSTRAINT unique_pk  
  EXPECT (num_entries = 1)  
) AS  
SELECT pk, count(*) as num_entries  
FROM report  
GROUP BY pk
```

GROUP BY detects primary key duplicates by counting occurrences.



DATABRICKS DATA PIPELINE
TABLE DEPENDENCIES & STATUS

DAG View in Lakeflow Interface

The Databricks UI automatically generates a visual DAG (Directed Acyclic Graph) of all tables in the pipeline. Each node shows status (running/success/failure), processing metrics, and data quality violations. You can click on a node to see the definition code, data preview, and detailed logs.

What the DAG shows

- Table dependencies (upstream → downstream)
- Real-time status of each table
- Row count and data volume
- Processing duration
- Summary of expectation violations

Interaction

- Clicking a node → table details
- Hovering over → quick metrics preview
- Timeline view → historical runs
- Detailed error analysis
- Schema evolution tracking

Standard Spark vs Lakeflow Pipelines

Feature	Standard Spark SQL/Python	Lakeflow Pipelines (DLT)
Programming model	SQL/DataFrame API, more boilerplate code.	Extended SQL/Python with DLT ('CREATE OR REFRESH', 'EXPECT').
Infrastructure automation	Manual cluster configuration, checkpoint management.	Automatic cluster management, checkpoint management, refresh.
Data quality	Custom implementation of data quality rules.	Built-in expectations ('EXPECT') with configurable reactions.
Monitoring and observability	Integration with external tools or custom implementation.	Automatic DAG, metrics, logs, and statuses in UI.
Dependency management	External orchestrator (e.g., Airflow).	Automatic detection and management of table dependencies.
Performance optimization	Manual optimization (e.g., partitioning, indexing).	Automatic Photon optimizations, caching, and scaling.

Automation – what do we get for free?

Data Lineage

Automatic table dependency graph indicates sources and consumers, propagating changes upstream.

State Management

Automatic checkpoint management and state processing, eliminating manual configuration and file tracking.

Incremental Refresh

The pipeline processes only changed data (deltas), saving resources and shortening execution time by 10–100 times.

Error Handling

Automatic retries and graceful degradation in case of errors, preventing pipeline hangs due to faulty records.

Observability

Built-in metrics (row count, processing time, data quality) available in UI, without additional code instrumentation.

Optimization

Automatic execution plan optimization and result caching, ensuring performance without manual intervention.

Compute

Lakeflow Pipelines offers two compute options, tailored to different data workloads.

Serverless

Build streaming and batch pipelines on a fully managed serverless platform requiring minimal additional configuration. Available in two modes –

Performance Optimized and **Standard**

\$0.45

/ DBU

Includes underlying compute costs

Serverless

Serverless for Lakeflow Pipelines is a fully managed option. It eliminates cluster configuration and the price is included in the DBU cost, with no additional cloud infrastructure fees. Two variants are available:

- **Standard:** up to 70% cheaper, start-up time 4–6 minutes.
- **Performance Optimized:** faster start-up (<1 minute), with Enzyme (incremental refresh), Stream Pipelining, and automatic scaling preventing OOM issues.

Both variants have the Photon engine automatically enabled, ensure automatic instance selection and scaling. They also support advanced features: CDC, SCD Type 2, and data quality rules.

Classic Clusters

Classic mode for Lakeflow Pipelines uses advanced clusters that require manual configuration. This variant involves separate payment for cloud instances and the cost of Lakeflow Pipelines DBU. Cluster startup time ranges from 1 to 6 minutes, and the Photon engine is optional. Please note that advanced features such as CDC, SCD Type 2, and data quality rules are unavailable in Classic Core.

Classic Core	Classic Pro	Classic Advanced
Easily build scalable streaming or batch pipelines in SQL and Python	Easily build scalable streaming or batch pipelines in SQL and Python and handle change data capture (CDC) from any data source	Easily build scalable streaming or batch pipelines in SQL and Python, handle change data capture (CDC) and maximize your data credibility with quality expectations and monitoring
\$0.30 / DBU	\$0.38 / DBU	\$0.54 / DBU
Plus underlying compute costs billed by cloud provider	Plus underlying compute costs billed by cloud provider	Plus underlying compute costs billed by cloud provider

Comparison: Classic vs Serverless

	Classic	Serverless Standard	Serverless Performance Optimized
Typical Use Case	Traditional workloads, full control	General data pipelines, lower costs	High-performance pipelines, fast start
Change Data Capture (CDC)	Not Available	Yes	Yes
Slowly Changing Dimensions (SCD) Type 2	Not Available	Yes	Yes
Data Quality Expectation Rules	Not Available	Yes	Yes
Photon Engine	Optional (2.5X DBU)	Automatically	Automatically
Automatic Instance Selection	Manual	Automatic	Automatic
Auto-scaling (OOM)	No	No	Yes
Enzym (Incremental Refresh)	No	No	Yes
Stream Pipelines	No	No	Yes
Startup Time	4-6 min	4-6 min	<1 min
Total Cost	Instances paid separately	Included in price (up to 70% cheaper)	Included in price

Refresh Techniques in Lakeflow Pipelines

Lakeflow Pipelines automatically manages the refreshing of materialized views. Below are the available refresh methods and their characteristics:

Technique	Incremental?	Description
FULL_RECOMPUTE	No	View fully recomputed from scratch.
NO_OP	N/A	View not updated (no changes in the base table).
ROW_BASED, PARTITION_OVERWRITE, WINDOW_FUNCTION, APPEND_ONLY, GROUP_AGGREGATE, GENERIC_AGGREGATE	Yes	View refreshed incrementally, minimizing resource usage and time.

Lakeflow Pipelines vs MLV (Fabric)

In the context of modern lakehouse architectures, Materialized Lake Views (MLV) and Lakeflow serve to automate data pipelines, but they differ in scope and platform.

Feature	Lakeflow Pipelines (Databricks)	Microsoft Fabric MLV
Ecosystem	Databricks (Unified Platform)	Microsoft Fabric (SaaS)
Scope	Entire data lifecycle (ingest, ETL, orchestration)	Single object/transformation
Language	SQL and Python	SQL (mainly)
Main Goal	Scalable, professional data pipelines	Fast data delivery, simplification of Medallion architecture
Mechanism	Automatic orchestration, incremental processing, autoscaling, real-time	Declarative approach – system manages refresh
Object Types	Streaming Tables and Materialized Views	Materialized Lake Views (Delta format in OneLake)
Data Quality	Built-in EXPECT (configurable reactions)	Built-in monitoring (e.g., CONSTRAINT ON MISMATCH)
CDC / SCD Type 2	Built-in syntax (APPLY CHANGES INTO)	Requires manual implementation
Monitoring	Automatic DAG, metrics, logs	Automatic lineage visualization, monitoring in Fabric

DEMO

Let's see Databricks in action!

Orchestration

Lakeflow Jobs

Where Does Workflows Fit in the Architecture?

Workflows is the **orchestration layer** — it manages scheduling, execution order, and error handling at the system level. It is the glue that connects all platform components into one coherent flow.

Scheduling and triggering

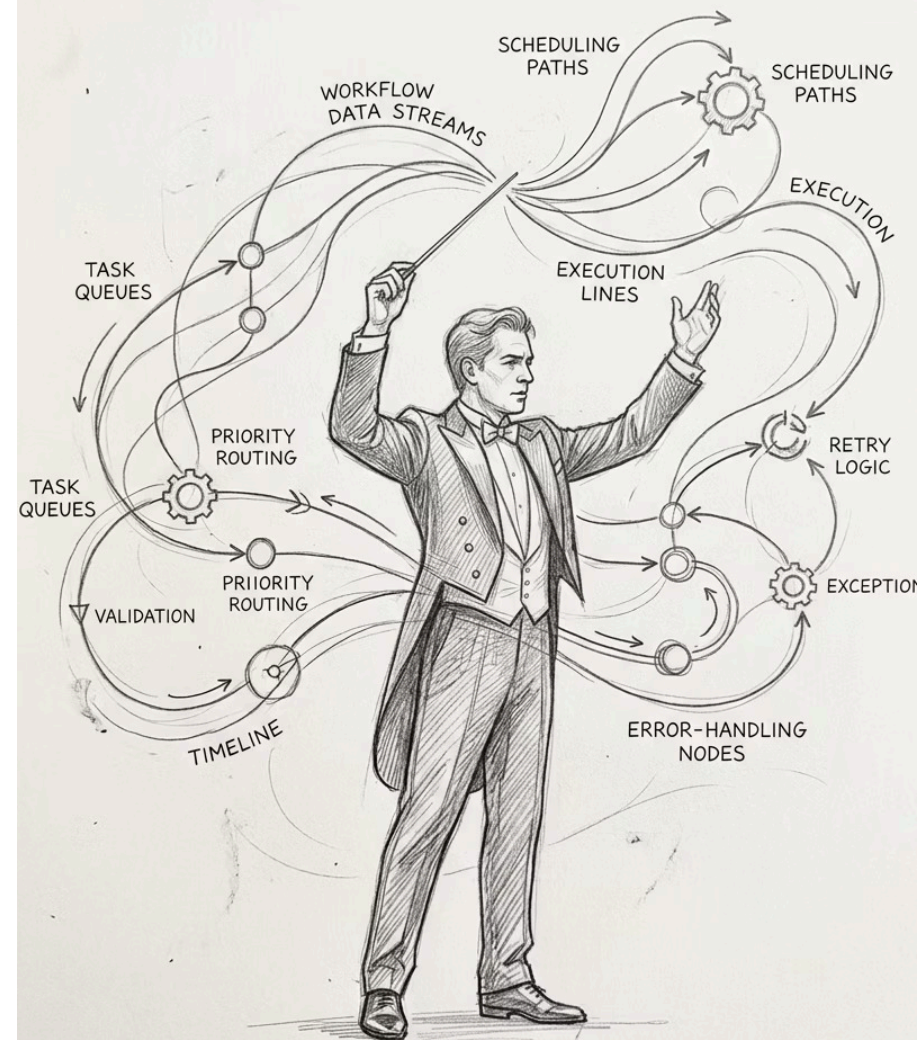
Running tasks at a specific time, on an event (file in S3, Kafka message), or after another task completes. Workflows supports both cron schedules and event triggers.

Managing execution order

Defining a task DAG within a single Workflow — which tasks can run in parallel and which must wait for previous ones. Real-time visibility into the status of each step.

Error handling and retry

Configuring retry policies, alerts, and notifications. Ability to define remediation tasks that are triggered automatically after an error is detected.



From Code to Production

Lakeflow Jobs (Databricks Workflows) is a managed orchestration platform that transforms notebooks and pipelines into production-grade processes. Zero external dependencies – everything in one place: scheduling, monitoring, alerting, retry logic.

Development

You write code in notebooks or Lakeflow pipeline definitions, testing locally or on a dev cluster

Job Definition

You define tasks, dependencies, compute requirements in the UI or via API

Scheduling

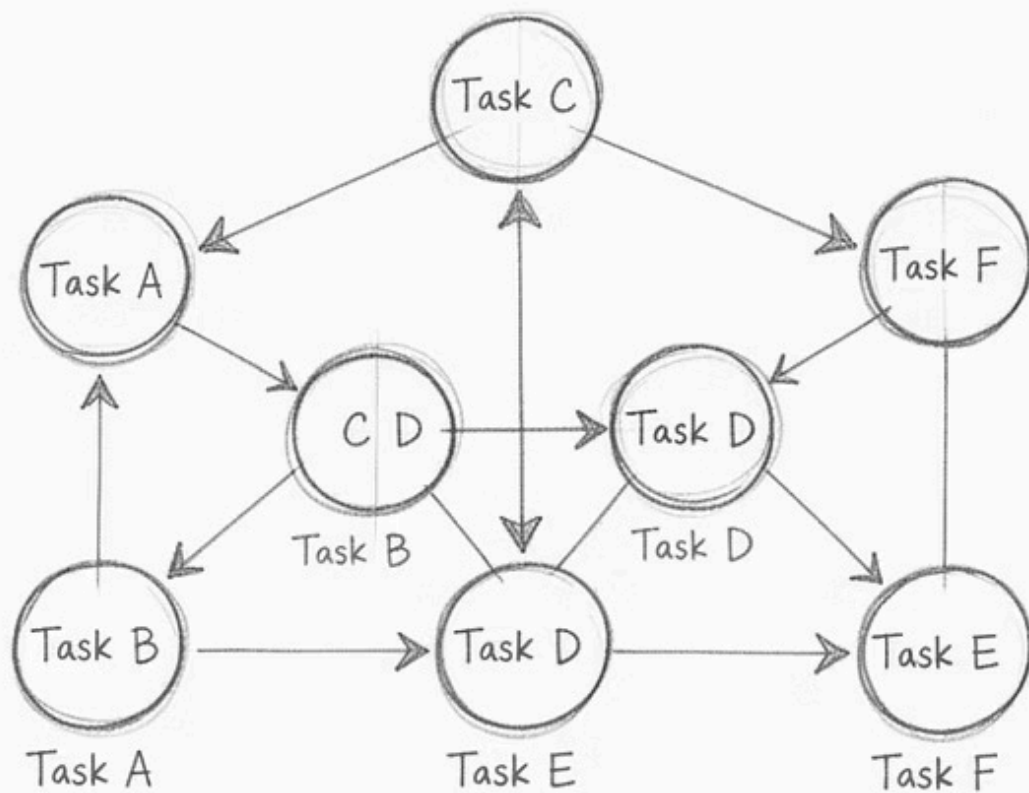
You configure the trigger: CRON schedule, file arrival, continuous, or manual

Execution

The job runs automatically, managing resources, retries, and notifications

Monitoring

You observe progress in real-time, analyze historical runs, and receive alerts



Job Structure – Tasks & Dependencies

A job consists of one or more tasks arranged in a DAG (Directed Acyclic Graph). Each task is an independent execution unit – it can be a notebook, Lakeflow pipeline, Python script, SQL query, or even a dbt transformation.

Task Types

Notebook Task

Runs a Databricks notebook with parameters. The most popular type for custom logic and exploratory workflows.

Lakeflow Pipeline Task

Executes an entire Lakeflow pipeline definition. Automatically propagates changes through the entire table DAG.

Python Task

Runs a standalone Python script (wheel file). Use for lightweight operations without the need for a notebook.

SQL Task

Executes an SQL query on an SQL Warehouse. Ideal for simple data quality checks or reporting queries.

JAR Task

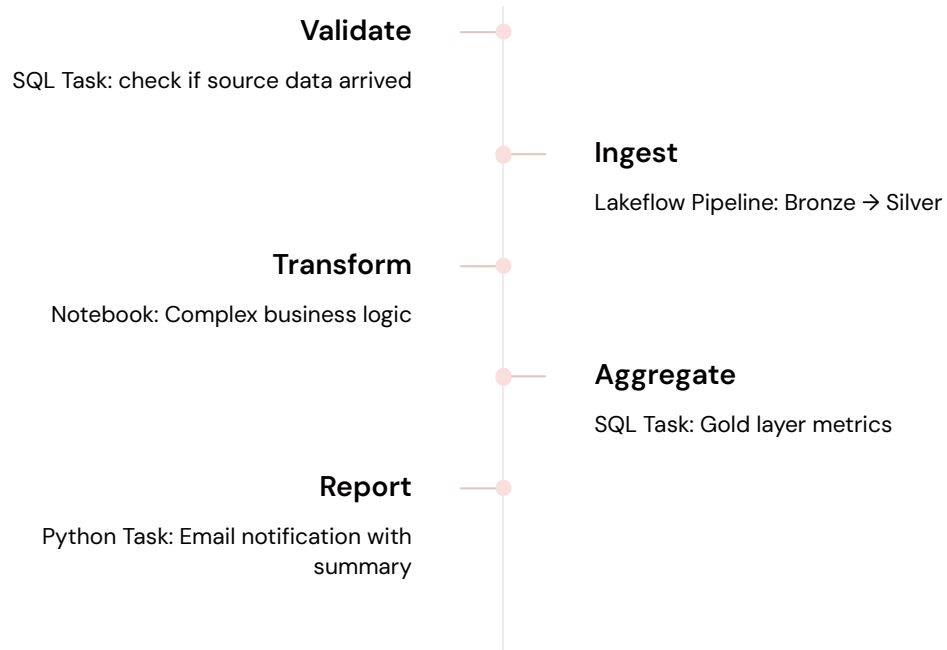
Runs a Scala/Java application. For legacy code or high-performance Spark jobs written in Scala.

Run Job Task

Invokes another job – enables composition and reuse. Modularization of complex workflows.

Example: Multi-stage ETL job

Data Flow



Dependency Configuration

```
# WJSON job definition:
{
  "tasks": [
    {
      "task_key": "validate", "sql_task": {...},
      {
        "task_key": "ingest",
        "pipeline_task": {...},
        "depends_on": [{"task_key": "validate"}]
      },
    {
      "task_key": "transform",
      "notebook_task": {...},
      "depends_on": [{"task_key": "ingest"}]
    },
    {
      "task_key": "aggregate",
      "sql_task": {...},
      "depends_on": [{"task_key": "transform"}]
    },
    {
      "task_key": "report",
      "python_wheel_task": {...},
      "depends_on": [{"task_key": "aggregate"}]
    }
  ]
}
```

Tasks execute in the order defined by dependencies. If validate fails, remaining tasks are skipped.

UI Demo: See live in Databricks Workflows UI – interactive DAG graph with real-time status of each task

Triggers – When to run a job?

Schedule (CRON)

A classic time-based trigger. You define when the job should run using a CRON expression or a simple UI.

Use case: Daily ETL at 2:00 AM, hourly metrics refresh, end-of-month reports

```
0 2 * * * (every day at 2:00 AM)
0 */4 * * * (every 4 hours)
```

File Arrival

An event-driven trigger that reacts to the appearance of new files in a specified location (S3, ADLS, DBFS).

Use case: Processing vendor files, partner data drops, automated imports

Reacts within <5 min of file arrival

Continuous

The job restarts immediately after the previous run finishes. For near-real-time processing.

Use case: Streaming pipelines, CDC processing, live dashboards

⚠ Beware of costs – the cluster runs non-stop

Manual / API

Run manually via UI, CLI, or REST API. For ad-hoc runs and integration with external orchestrators.

Use case: One-off data loads, testing, integration with Airflow/Azure Data Factory

```
databricks jobs run-now --job-id 123
```

Table Update

Trigger that runs when a specific Delta table is updated. Monitors table changes and automatically starts the job.

Use case: Downstream processing triggered by upstream table updates. Event-driven pipelines that react to data availability.

Example: When the bronze layer is refreshed, automatically trigger silver layer transformation.

Types of Parameterization

Job Parameters

Global for the entire job, available in all tasks:

```
# W job config:
"parameters": {
  "environment": "prod",
  "date": "2024-01-15"
}

# W notebook:
env =
dbutils.widgets.get("environment")
```

Task Parameters

Specific to a single task, override job params:

```
"notebook_task": {
  "notebook_path": "/ETL/transform",
  "base_parameters": {
    "input_table": "silver.orders",
    "output_table": "gold.daily_metrics"
  }
}
```

Task Values

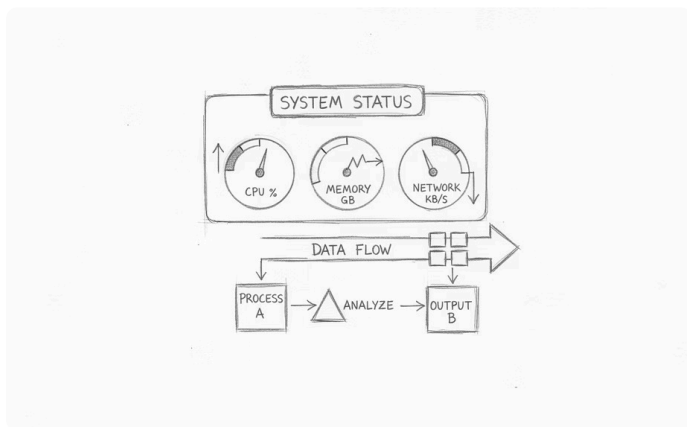
Passing data between tasks – the output of one is the input of the next:

```
# Task 1 - writes result:
dbutils.jobs.taskValues.set(
  key="row_count",
  value=df.count()
)

# Task 2 - reads:
count = dbutils.jobs.taskValues.get(
  taskKey="task_1",
  key="row_count"
)
```

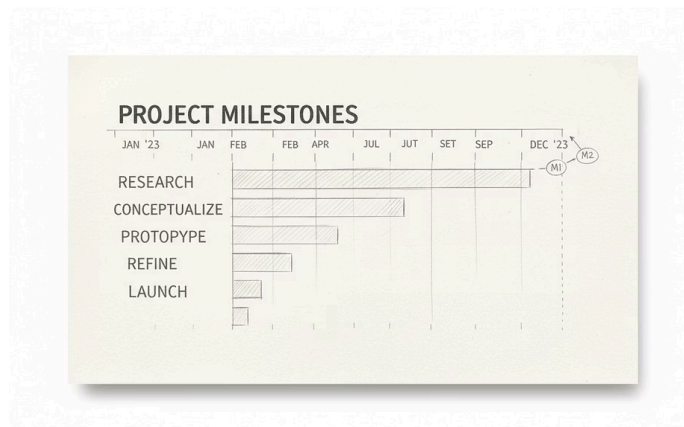
Observability – Monitoring and Alerting

Databricks provides multi-layer observability: job-level metrics, task-level details, cluster utilization, and query-level performance. Everything out of the box, zero external tools required.



Job Run History

Each run is saved with full context: duration, status, parameters, cluster config, output logs. Historical trends and comparisons between runs.



Task Timeline

Gantt chart showing the execution of each task, dependencies, bottlenecks. Identify which tasks are slow and optimize accordingly.



Alerting

Email, Slack, PagerDuty, webhook notifications for job failure, timeout, or success. Customizable thresholds and retry logic.

System Tables for Advanced Analytics

Databricks records all job execution metadata in System Tables – managed Delta tables that you can query with SQL. This allows for custom reporting, trend analysis, cost attribution, and SLA monitoring.

Key Tables

- **system.lakeflow.job_run_timeline:** History of all job runs with duration, status, cost
- **system.lakeflow.task_run_timeline:** Task-level breakdown of each run
- **system.compute.clusters:** Cluster usage and uptime
- **system.billing.usage:** DBU consumption per job/task

Example query

```
-- Top 10 longest job runs
SELECT
  job_name,
  run_id,
  start_time,
  ROUND(run_duration_sec/60, 2) as duration_min,
  result_state
FROM system.lakeflow.job_run_timeline
WHERE start_time >= current_date() - 7
ORDER BY run_duration_sec DESC
LIMIT 10
```

You can transform these queries into dashboards in Databricks SQL, scheduled reports, or alerts (e.g., "notify if any job runs >2h")

Notebook: Notebook 03, section "Observability with System Tables"

Orchestration Patterns — Trade-offs

Choosing an orchestration pattern has a direct impact on flexibility, operational complexity, and response time to errors. There is no single right pattern — there are trade-offs you need to choose consciously.

1

Sequential

Bronze → Silver → Gold as a linear chain. Simplicity and predictability. Slower — the next step waits for the previous one to finish. Good for small volumes and simple pipelines.

2

Event-driven

Each layer triggers the next after successful completion. Better fault tolerance and isolation. Complex dependency configuration. Good for pipelines with varied processing times.

3

Layer-based

Separate Workflows for Bronze, Silver, and Gold with clear synchronization points. Independent scaling and deployment of each layer. Requires state management between Workflows. Good for large, complex platforms.

Common Anti-Patterns in Production Pipelines

Recognizing anti-patterns is half the battle in eliminating them. The following issues appear regularly in Databricks platform audits — regardless of the team's experience.

✗ Mixing ingestion with transformation

An Auto Loader notebook contains business logic. Result: it is impossible to reprocess data without reloading it from the source. Fixed by: a dedicated Bronze layer with no transformations.

✗ Duplicating logic across layers

The same deduplication rule is implemented in Silver and in each Gold table separately. Result: inconsistent outcomes and multiple places to update when the rule changes.

✗ Overusing unstructured notebooks

One notebook = one pipeline. Hundreds of lines of code without modularization, without tests, without documentation. Result: code that is impossible to maintain after 6 months.

✗ Treating Medallion as a complete architecture

Bronze/Silver/Gold is a data storage layering pattern — not a system architecture. Without orchestration, dependency management, and separation of concerns, it is just three folders in DBFS.

Key Takeaways

Designing production data pipelines is systems engineering — not scripting. Every architectural decision has long-term consequences for maintainability, scalability, and platform reliability.



Architecture over tools

Tool knowledge is the starting point. You create value by thoughtfully combining them into a system with a clear structure.



Pipeline as a system, not a script

Design with dependencies, failures, and changing requirements in mind — not just the happy execution path.



Clear responsibility boundaries

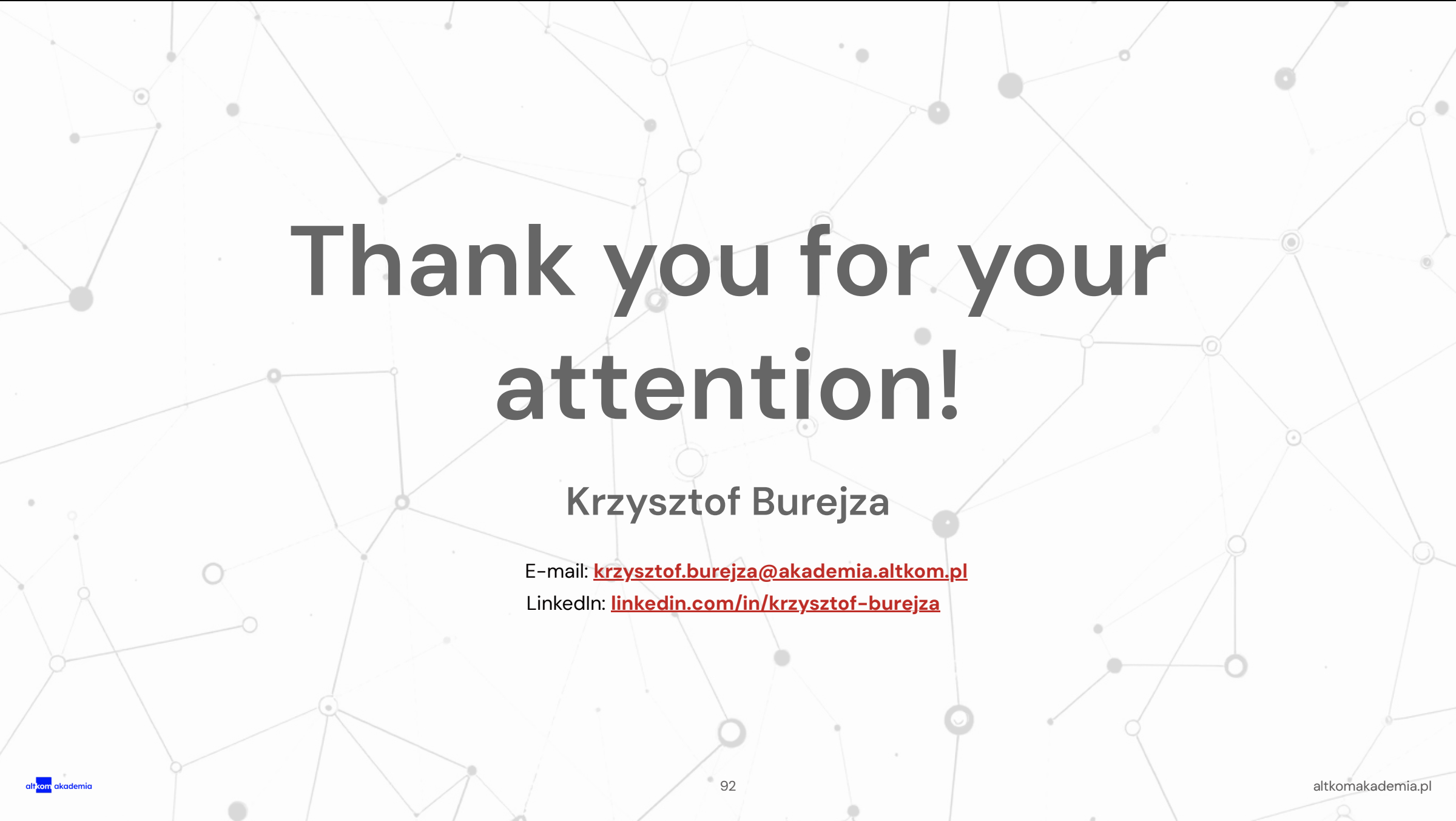
Each layer does one thing. Mixing responsibilities is the fastest path to systems that are impossible to maintain.



Modularity and reusability

Build components that you can test and deploy independently. Avoid logic duplication — centralize and import.

🟢 Goal: data platforms that run reliably, scale without redesign, and can be extended by new team members without weeks of onboarding.



Thank you for your attention!

Krzysztof Burejza

E-mail: krzysztof.burejza@akademia.altkom.pl

LinkedIn: [linkedin.com/in/krzysztof-burejza](https://www.linkedin.com/in/krzysztof-burejza)